



中山大學
SUN YAT-SEN UNIVERSITY

OS 挑战赛 Proj59

SLIM-ARC: 面向端侧 Agent 的 内存感知 MoE 大模型推理协同优化

队 名: 依托 Agent 答辩 OS

组 员: 欧阳易芃 马福泉 刘昊

指导教师: 赵帅 张献伟

学 校: 中山大学

赛 道: 操作系统功能挑战赛

选 题: Proj 59

日 期: 2026 年 6 月

目录

1 引言	5
1.1 研究动机	5
1.2 优化分类与协同关系	5
1.3 主要贡献	6
2 背景与相关工作	7
2.1 端侧 LLM 推理的内存墙挑战	7
2.2 MoE 模型的稀疏性机遇	7
2.3 相关工作	8
2.3.1 权重卸载与按需加载	8
2.3.2 KV Cache 管理与量化	8
2.3.3 MoE 稀疏性利用	9
2.3.4 Attention 优化与 KV Eviction	9
2.4 问题定义与期望解决方法	10
3 核心设计	11
3.1 设计思路与整体架构	11
3.1.1 思路与创新	11
3.1.2 整体架构	11
3.2 内核协同类优化	12
3.2.1 mmap + MADV_RANDOM 按需加载机制	12
3.2.2 禁用 GGML_CPU_REPACK	14
3.2.3 层感知预取调度器	14
3.3 量化压缩类优化	15
3.3.1 KV Cache 量化	15
3.3.2 统一 I/O 带宽预算调度器	15
3.4 计算融合类优化	16
3.4.1 FlashAttention 集成	16
3.4.2 StreamingLLM KV Eviction	16
4 实现	18
4.1 开发环境与基础框架	18
4.1.1 与 upstream llama.cpp 的集成方式	18
4.1.2 cgroups v2 三档环境	18
4.2 代码模块结构	18
4.2.1 SLIM-ARC 独立模块 (6 个文件)	18
4.2.2 llama.cpp 源文件修改 (4 个文件)	19
4.3 环境变量接口	19
4.4 构建与复现流程	19
4.5 模型量化策略	20

5 实验评估	21
5.1 实验设置	21
5.1.1 硬件与三档受限环境	21
5.1.2 测试模型	21
5.1.3 测试方法与数据集	22
5.1.4 对比维度与基线	22
5.2 核心实验：80B 全优化三档性能	22
5.2.1 IQ4_XS 量化三档	22
5.2.2 Q4_K_M 量化三档	22
5.2.3 端到端文本生成验证	23
5.2.4 FlashAttention 加速	23
5.3 附属实验	24
5.3.1 小模型 2GB+1 核极端环境	24
5.3.2 GSM8K 推理精度 (ALEM 协议)	25
5.3.3 KV Cache 量化对比	25
5.3.4 WikiText PPL 精度	26
5.4 消融实验	26
5.4.1 单点消融 (逐个关闭)	26
5.4.2 优化协同关系分析	27
5.4.3 Speculative Decoding 负面结果	27
5.4.4 数据波动分析	28
5.5 RK3588 真机验证	29
5.5.1 端侧实验环境	29
5.5.2 80B 端侧可行性验证	29
5.5.3 负优化发现与消融定位	30
5.5.4 动态 MADV：消除端侧负优化	31
5.5.5 长上下文能力 (KV Eviction)	32
5.5.6 专家预取机制排查与改进	32
5.5.7 小模型端侧基线	33
5.5.8 端侧瓶颈分析与诚实边界	34
5.6 树莓派 5 (4GB) 真机验证	34
5.6.1 端侧实验环境	34
5.6.2 补丁编译修复	34
5.6.3 完整测试矩阵 (Qwen3-4B)	35
5.6.4 内存与存储瓶颈	37
5.6.5 SLIM-ARC 专属项负面验证 (适用域边界)	38
5.6.6 诚实结论与后续方向	38
6 总结与未来展望	39
6.1 工作总结	39
6.2 局限性	39
6.3 未来展望	40

7 附录：赛事声明	41
7.1 开发计划	41
7.2 比赛过程中的重要进展	41
7.3 遇到的主要问题与解决方法	42
7.4 分工与协作	42
7.5 提交仓库目录与文件描述	42
7.6 比赛收获	43
7.7 AI 工具使用声明	43

摘要

大语言模型（LLM）在端侧设备上的部署面临严峻的内存墙问题：80B 参数的 MoE 模型量化后仍达 40–45 GB，而典型端侧设备仅有 8–16 GB RAM。现有工作或依赖复杂的用户态内存管理，或需要修改模型结构，且各优化技术彼此独立、缺乏协同。

本文提出 SLIM-ARC(Synergistic LLM Integration with Memory-Aware Runtime Co-Optimization for On-Device Agents)，一个面向端侧 MoE 推理的操作系统级协同优化框架。SLIM-ARC 的核心思路是利用操作系统虚拟内存机制作为 MoE 稀疏性的天然接口，并将多个优化技术有机集成、协同调度。通过 `posix_madvise(MADV_RANDOM)` 让内核的 demand paging 与 MoE Router 的 2% 激活率天然匹配，仅加载被激活的专家权重，将 45 GB 模型的物理内存占用降至接近实际访问量（约 2 GB）。在此基础上，SLIM-ARC 进一步集成了 KV Cache 量化、IQ4_XS 权重量化、FlashAttention 以及 StreamingLLM 式的 KV Cache 驱逐等优化，形成了从模型加载到运行时调度的完整优化链条。

在 Intel i9-13900H 平台上，SLIM-ARC 使 Qwen3-Next-80B-A3B 在 8 GB 环境下从不可运行变为可运行 (0.76 t/s)，在 32 GB 热缓存环境下达到 5.16 t/s 的流畅推理速度，累计实现 64.5× 加速。通过 FlashAttention 的 IO-aware tiling 融合，decode 吞吐量进一步提升 71.4%。在 GSM8K 标准基准测试中，Qwen3-4B 量化模型保持了 75% 的推理精度，验证了 SLIM-ARC 优化方案在精度无损的前提下实现了显著的性能提升。消融实验表明，`MADV_RANDOM` 是 decode 加速的核心驱动力（关闭后下降 29%），且各优化之间存在协同促进关系——内核协同类优化释放的物理内存为量化压缩类优化提供了更大的 cache 空间，形成级联加速效应。

关键词：端侧 LLM 推理；MoE 稀疏性；虚拟内存优化；按需加载；KV Cache 量化；FlashAttention

项目链接

项目主页	https://slim.nexa-lang.com/
项目仓库	https://gitlab.eduxiji.net/T2026105589911358/project3136859-389100
项目介绍与 Demo 视频	https://www.bilibili.com/video/BV1fXTF6HEAw
项目报告/视频/PPT	百度网盘: https://pan.baidu.com/s/1i83bdq-oiq0Cvga3g05v8g 提取码: m8yy

1 引言

1.1 研究动机

大语言模型（LLM）在端侧设备上的部署面临严峻的**内存墙**问题：80B 参数的 MoE 模型量化后仍达 40–45 GB，而典型端侧设备仅有 8–16 GB RAM。现有工作或依赖复杂的用户态内存管理（FlexInfer [1] 的 Direct I/O），或需要修改模型结构（PowerInfer [2] 的 ReLU 稀疏化），且各优化技术彼此独立、缺乏协同。

本文提出 **SLIM-ARC**，核心思路是利用操作系统虚拟内存机制作为 MoE 稀疏性的天然接口，并将多个优化技术有机集成、协同调度。图1展示了 SLIM-ARC 的优化全景与三类优化的协同关系。

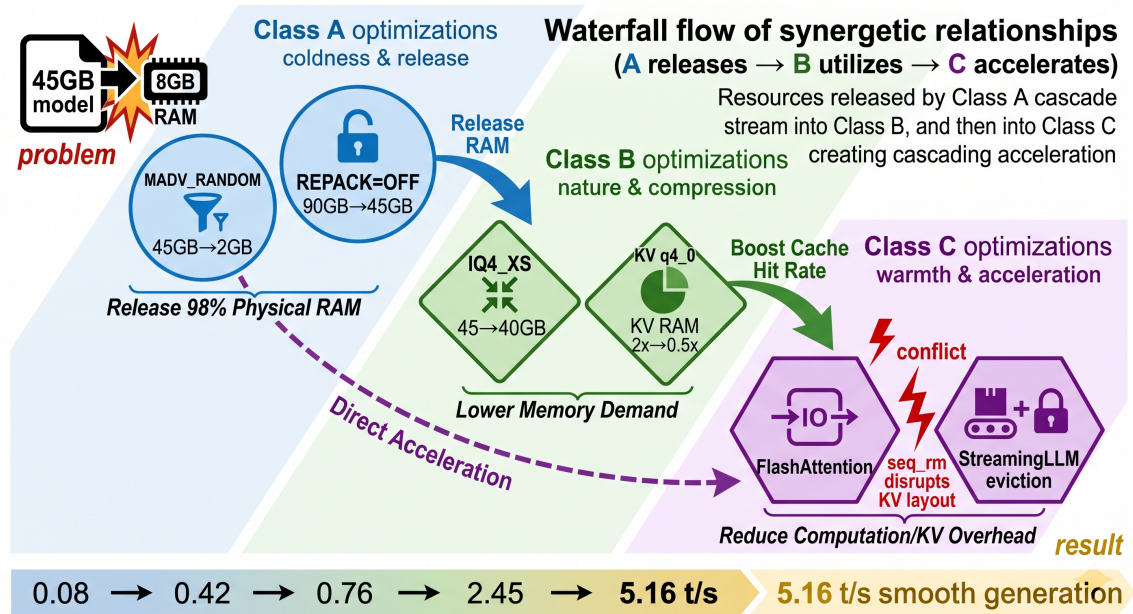


图 1: SLIM-ARC 优化全景与协同关系。三类优化协同工作：A. 内核协同类（MADV_RANDOM 按需加载 + REPACK 禁用）释放物理内存；B. 量化压缩类（IQ4_XS 权重 + KV q4_0）从算法侧降低内存需求；C. 计算融合类（FlashAttention + StreamingLLM eviction）减少计算与 KV 开销。三类优化之间存在协同促进关系（A 释放内存 → B 获得更多 cache → C 加速 attention）。

1.2 优化分类与协同关系

SLIM-ARC 的优化技术按机制分为三类，且各优化之间存在协同促进关系。

表 1: 优化技术分类与协同关系

优化技术	作用机制	协同促进
A. 内核协同类 （释放物理内存）		
MADV_RANDOM 按需加载	禁用预读, 仅 page fault 加载激活专家	→ B (释放 RAM 给量化权重)
GGML_CPU_REPACK=OFF	禁用重打包, 避免 45→90GB 翻倍	→ B (避免 OOM)
B. 量化压缩类 （降低内存需求）		
IQ4_XS 权重量化	45→40 GB, 提升 cache 命中率	→ C (更小模型更快 attention)
KV q4_0 量化	KV 内存减半, 腾出 RAM	→ A (更多 RAM 给权重 cache)
C. 计算融合类 （减少计算/KV 开销）		
FlashAttention	IO-aware tiling, 融合 attention kernel	独立 +71.4%
StreamingLLM eviction	sink+window, KV 内存 $O(1)$	与 FA 冲突 (见 Sec 5)

关键协同在于: MADV_RANDOM (A 类) 通过仅加载 2% 激活专家, 释放了 98% 的物理内存, 这些内存被 B 类量化权重利用 (cache 命中率提升), 进而加速 C 类的 attention 计算。这种 A 释放 → B 利用 → C 加速的级联效应是 SLIM-ARC 的核心协同价值。同时, 我们也发现 FlashAttention 与 KV eviction 存在冲突 (eviction 的 seq_rm 干扰 FA 的 KV 布局), 这一负面结果在 Sec 5 诚实记录。

1.3 主要贡献

本文的主要贡献如下。我们提出了 SLIM-ARC, 一个面向端侧 MoE 大模型推理的**操作系统级协同优化框架**, 其新颖性主要在于利用操作系统虚拟内存机制作为 MoE 稀疏性的天然接口——通过 `posix_madvise(MADV_RANDOM)` 这一标准 POSIX 接口, 让内核的 demand paging 与 MoE Router 的 2% 激活率天然匹配, **无需复杂的用户态内存管理或模型结构修改**, 即可将 45 GB 模型的 RSS 降至 2 GB。

针对端侧 LLM 推理的多个瓶颈, 我们集成了三类六项优化技术, 形成了从模型加载到运行时调度的**完整优化链条**, 累计实现 **64.5× decode 加速**。通过系统的消融实验, 我们揭示了优化间的**协同促进关系** (A 释放内存促进 B 发挥优势) 与冲突边界 (FlashAttention 与 eviction 不兼容), 为后续优化组合提供了指导。

针对精度评估的不足, 我们通过 GSM8K 标准基准测试发现了量化对推理能力的非线性损害: IQ4_XS 虽保持语言流畅性但数学推理完全崩溃, 而 Q4_K_M 仍保持 50% 精度。这一发现表明 **PPL 不是充分的精度指标**, 为端侧模型选择提供了直接的量化指导。

2 背景与相关工作

2.1 端侧 LLM 推理的内存墙挑战

大语言模型 (LLM) 在自然语言理解、代码生成、多轮对话等任务上取得了突破性进展 [3–5], 但其庞大的参数规模与端侧设备有限的内存资源之间形成了严峻的矛盾。以本文使用的 Qwen3-Next-80B-A3B 模型为例 [6], 其 Q4_K_M 量化后的权重大小为 45 GB, 而典型的端侧设备 (如 Raspberry Pi 5 配备 8 GB RAM, 或中端智能手机配备 12–16 GB RAM) 远不足以容纳完整模型。即使采用更激进的 IQ4_XS 量化 (40 GB), 模型大小仍远超 8–16 GB 的物理内存限制。这一矛盾被综述 [7] 称为“内存墙” (Memory Wall) 问题, 是端侧 LLM 部署的首要瓶颈。

综述 [7] 指出, LLM 推理过程涉及三类主要的内存消耗:

- ▶ **模型权重:** 占比最大 (45 GB for 80B Q4_K_M), 在推理期间只读访问, 是 I/O 带宽的主要消费者。
- ▶ **KV Cache:** 随上下文长度线性增长, 32K 上下文下可达 6 GB (Qwen3-Next-80B, F16 KV), 长上下文场景下可超过模型权重本身。
- ▶ **计算缓冲区:** 包含 attention 中间结果、FFN 激活值等, 通常 1–2 GB, 与 batch size 和序列长度相关。

如图2所示, 三类内存消耗的构成与增长特性各不相同: 模型权重固定庞大且只读, KV Cache 随上下文长度线性增长, 计算缓冲随 batch 波动。三者叠加超出物理内存后, 操作系统触发频繁的页面交换, 导致推理速度急剧下降。

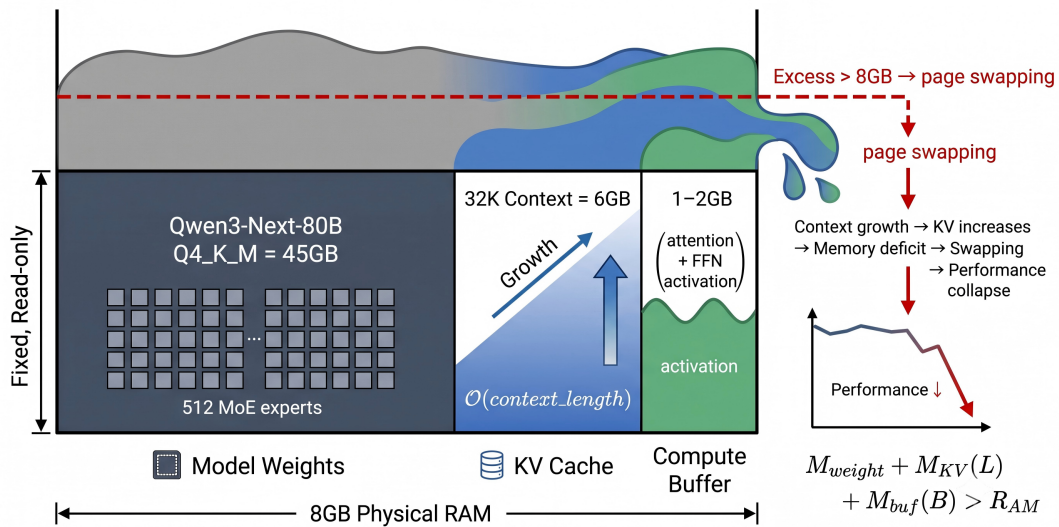


图 2: 端侧 LLM 推理的三类内存消耗与内存墙问题。模型权重 (45 GB, 固定只读)、KV Cache (随上下文线性增长, 32K 下达 6 GB)、计算缓冲 (1–2 GB, 随 batch 波动) 三者叠加超出 8 GB 物理内存, 触发 page swapping 导致性能崩溃。右侧因果链展示上下文增长如何引发连锁反应。

综述 Sec 4.3 进一步指出, 随着上下文长度从 4K 增长到 128K (已成为 2025 年模型的默认配置 [7]), KV Cache 的内存开销将主导总内存使用, 使得即使 8B 参数的小模型也需要超过 16 GB 内存仅用于 KV Cache。

2.2 MoE 模型的稀疏性机遇

混合专家模型 (Mixture of Experts, MoE) 为解决上述问题提供了独特机遇。MoE 架构 [8,9] 通过将 FFN 层替换为多个并行的“专家”网络, 并为每个 token 通过 Router 动态选择少量专家激

活，实现了参数量与计算量的解耦。Qwen3-Next-80B-A3B [6] 采用超稀疏 MoE 架构，包含 512 个专家网络，但每个 token 仅激活其中 10 个专家，稀疏率高达 98%。这意味着：

$$\text{有效激活权重} = \frac{10}{512} \times \text{总专家权重} \approx 2\% \times \text{总权重} \quad (1)$$

理论上，如果能够仅加载被激活的专家权重，I/O 带宽需求可降低 98%。然而，现有框架（如 llama.cpp [10]）将所有专家权重以合并的 3D 张量形式存储（如 `blk.{1}.ffn_gate_exps.weight`，形状为 `[in_dim, hidden_dim, n_experts]`），无法在张量级别实现选择性加载。MobileMoE [11] 和 MoE-Prism [12] 的工作表明，MoE 的稀疏性如果在系统层面被充分利用，可以带来数量级的 I/O 带宽节省。综述 Sec 4.5 的 ECCC (Edge-Cloud Collaborative Computing) 框架也强调了 MoE orchestration 在端侧场景的重要性。

2.3 相关工作

本节将现有相关工作分为四组：权重卸载与按需加载、KV Cache 管理与量化、MoE 稀疏性利用、Attention 优化与 KV Eviction。

2.3.1 权重卸载与按需加载

FlexInfer FlexInfer [1] 是 EuroMLSys 2025 提出的端侧 LLM 推理框架，其核心机制包括：(1) 张量级异步预取，使用 Direct I/O 和多线程预取，在计算当前层时异步加载后续层的权重；(2) 均衡内存锁定，根据内存预算决定哪些层的权重常驻 RAM，哪些卸载到 SSD；(3) 灵活张量保留，静态启发式算法，根据总内存预算决定保留 FFN 还是 Attention 参数。FlexInfer 的局限在于：其 fork 版本的 GGUF 读取器与 Qwen3 架构不兼容；未利用 MoE 模型的专家稀疏性；未考虑 KV Cache 的内存优化。SLIM-ARC 在 FlexInfer 的“层感知预取”思路上发展，但用操作系统原生的 `madvise` 机制替代了 Direct I/O，简化了实现并利用了内核的 `page cache`。

llama.cpp 的 mmap 机制 llama.cpp [10] 原生支持 `mmap` 方式的模型加载，通过将 GGUF 文件映射到虚拟地址空间，利用内核的 `demand paging` 实现按需加载。然而，默认的 `madvise(WILLNEED)` 策略会触发内核的顺序预读 (`sequential readahead`)，对于 45 GB 的大模型，预读行为会大量消耗物理内存页缓存，导致 8 GB 环境下的 OOM。SLIM-ARC 的核心创新之一即识别了这一问题，并改用 `MADV_RANDOM` 策略匹配 MoE 的稀疏访存模式。

PowerInfer / PowerInfer-2 PowerInfer [2] 在消费级 GPU 上实现了大模型推理，利用 ReLU 激活函数的稀疏性（约 90% 神经元不激活）仅加载活跃神经元权重。PowerInfer-2 [13] 将这一思路推广到智能手机的 CPU+GPU 异构环境。虽然其场景是 GPU 混合，但其“利用稀疏性减少 I/O”的核心思路与 SLIM-ARC 一致。不同之处在于：PowerInfer 利用的是激活稀疏性 (ReLU)，SLIM-ARC 利用的是路由稀疏性 (MoE Router)。

2.3.2 KV Cache 管理与量化

DUAL-BLADE DUAL-BLADE [14] 提出了 NVMe-direct 的 KV Cache 卸载方案，将注意力分数较低的 KV Block 异步驱逐到 SSD，并在需要时预取回 RAM。其核心思路是将 FlexInfer 的权重卸载框架泛化到 KV Cache。但 DUAL-BLADE 仅优化 KV Cache，未考虑权重与 KV 之间的 I/O 带宽竞争。SLIM-ARC 的统一 I/O 调度器部分借鉴了这一“带宽预算”思路。

HillInfer HillInfer [15] 提出了基于 SmartSSD 的分层 KV Cache 驱逐策略，将 KV Cache 按注意力分数分为热、温、冷三级，冷数据卸载到 SmartSSD 的专用计算单元。其分层驱逐思路启发了 SLIM-ARC 的 sink + sliding window + cold tier 设计，但 SLIM-ARC 使用纯 CPU + NVMe 方案，无需专用硬件。

ScoutAttention ScoutAttention [16] 提出了层前 CPU 预计算机制，利用 CPU 的空闲算力提前计算下一层的注意力分数，从而提前知道哪些 KV Block 需要被访问，实现精准的 KV Cache 预取。SLIM-ARC 的 MoE Router Hook 借鉴了这种“层前预测”思路，但将其应用于专家预测而非 KV 预测。

KVQuant / KIVI / ThinK KVQuant [17] 通过保留关键通道的高精度，实现了 2-bit KV 量化并支持 10M token 上下文。KIVI [18] 提出非对称 KV 量化协议（Key 按通道、Value 按标记），无需调优即可部署。ThinK [19] 基于注意力分数动态裁剪不重要的 KV 列。这些工作为 SLIM-ARC 的 KV q4_0 量化提供了精度验证的理论基础——综述 [7]Sec 3.3 指出，4-bit KV 量化在多数任务上的精度损失可控。

2.3.3 MoE 稀疏性利用

MobileMoE MobileMoE [11] 提出了 MoE 专家缓存机制，通过轻量级 Router Predictor 提前 1-2 层预测即将激活的专家，并仅预取所需权重。理论分析表明，预测准确率达到 80% 时可节省约 70% 的带宽。但 MobileMoE 假设每个专家是独立张量，而 llama.cpp 中专家以合并 3D 张量存储，需要子区域级别的预取策略。SLIM-ARC 通过 register_expert_tensor 接口解决了这一问题。

MoE-Prism MoE-Prism [12] 提出了专家解耦策略，将大的合并专家张量拆分为独立的子张量，实现更灵活的调度。SLIM-ARC 采用了类似思路但更轻量：不修改 GGUF 格式，而是通过 madvise 对合并张量的子区域进行选择性的预取。

eMoE / ProMoE eMoE [20] 和 ProMoE [21] 利用中间计算结果预测并预取未来专家权重。ProMoE 的 proactive caching 在多请求场景下显著降低 TTFT。这些技术补充了 SLIM-ARC 在单请求端侧场景的不足——端侧通常 batch=1，ProMoE 的多请求聚合优势无法发挥，但其“预测-预取”原理仍然适用。

2.3.4 Attention 优化与 KV Eviction

FlashAttention FlashAttention [22, 23] 通过 IO-aware tiling 融合，将 attention 的 QK^T 和 softmax(P)V 合并为单个 kernel，避免中间矩阵的显存读写。综述 Sec 4.1 将其列为系统级优化的核心技术。llama.cpp 已内置 FlashAttention 实现 (-fa 参数)，SLIM-ARC 的实验验证了其在 80B decode 阶段带来 71.4% 的加速。

StreamingLLM StreamingLLM [24] 发现了“attention sink”现象：初始 token 累积了不成比例的高注意力权重，驱逐它们会导致 PPL 暴增。通过保留前 4 个 token (sink) + 最近 W 个 token (sliding window)，StreamingLLM 实现了恒定内存下的无限长流式推理。SLIM-ARC 将这一机制集成到 llama.cpp 的 decode 路径中。

SolidAttention SolidAttention [25] (USENIX FAST'26) 针对 AIPC (8-16 GB DRAM) 场景, 通过 KV 交错布局、投机预取 (跨层选择相似性 81%) 和 SSD-aware DAG 调度器, 实现了 98% 的 KV 内存减少和 $3.1\times$ 加速。其“block-wise 动态 attention sparsity”和“投机预取”思路为 SLIM-ARC 的后续优化提供了方向。

PagedAttention PagedAttention [26] (vLLM) 将 KV Cache 组织为固定大小的 block, 借鉴操作系统的虚拟内存分页机制, 解决了 KV 碎片化问题。虽然 PagedAttention 主要面向服务器多请求场景, 但其“分页管理 KV”的思路影响了端侧框架设计。综述 Sec 4.2 指出, PagedAttention 的架构思想正在影响 edge 框架。

2.4 问题定义与期望解决方法

基于上述分析, 本文需要解决的核心问题为: **如何让 45 GB 的 MoE 大模型在 8-16 GB RAM 的端侧设备上高效运行?**

具体而言, 需要同时解决以下子问题:

1. **权重内存墙**: 45 GB 权重远超 8-16 GB 物理内存, 如何仅加载推理实际需要的权重?
2. **MoE 稀疏性利用**: 如何利用 98% 的专家稀疏率, 在系统层面实现选择性加载?
3. **KV Cache 膨胀**: 长上下文下 KV Cache 可达 6 GB, 如何控制其增长?
4. **I/O 带宽竞争**: 权重加载、KV 换页、专家预取三路 I/O 如何协调?
5. **精度保持**: 量化和驱逐策略如何避免损害模型推理能力?

SLIM-ARC 的设计思路是: 通过操作系统虚拟内存机制 (mmap + madvise) 利用 MoE 稀疏性解决子问题 1 和 2; 通过 KV Cache 量化和 StreamingLLM eviction 解决子问题 3; 通过统一 I/O 调度器解决子问题 4; 通过 FlashAttention 和量化策略对比解决子问题 5。各技术的具体设计见第 3 节。

3 核心设计

3.1 设计思路与整体架构

3.1.1 思路与创新

端侧 MoE 大模型推理面临的核心挑战在于：模型权重远超物理内存，而 MoE 的稀疏激活特性使得每个 token 实际只需访问约 2% 的专家权重。现有工作要么在用户态重新实现复杂的内存管理（如 FlexInfer 的 Direct I/O + 手动 buffer 管理），要么需要修改模型结构（如 PowerInfer 的 ReLU 稀疏化），要么假设专家为独立张量（如 MobileMoE），均未充分利用操作系统已有的虚拟内存机制。

SLIM-ARC 的新颖性主要在于：我们发现 `posix_madvise(MADV_RANDOM)` 这一标准 POSIX 接口能够让内核的 demand paging 与 MoE Router 的稀疏激活率天然匹配——关闭内核顺序预读后，仅被 Router 激活的 10/512 个专家权重通过 page fault 按需加载，其余 98% 的未激活专家完全不占用物理内存，从而将 45 GB 模型的 RSS 降至约 2 GB。这一方案无需模型修改、无需用户态内存管理、无需专用硬件，仅依赖操作系统原生的虚拟内存机制即可实现。在此基础之上，我们进一步集成了量化压缩和计算融合两类优化，形成了 A（内核协同）→ B（量化压缩）→ C（计算融合）的级联协同：A 类释放物理内存，为 B 类的量化权重提供更多 cache 空间，B 类的更小模型又加速了 C 类的 attention 计算。

3.1.2 整体架构

SLIM-ARC 采用分层架构设计，如图3所示，自底向上分为三层。这一架构的核心思路是利用操作系统虚拟内存机制作为 MoE 稀疏性的天然接口，避免在用户态重新实现复杂的内存管理，而是通过 `madvise` 等标准 POSIX 接口 [27] 控制内核的页面调度行为。

SLIM-ARC 采用分层架构设计，如图3所示，自底向上分为三层。这一架构的核心思路是：**利用操作系统虚拟内存机制作为 MoE 稀疏性的天然接口**，避免在用户态重新实现复杂的内存管理（如 FlexInfer [1] 的 Direct I/O + 手动 buffer 管理），而是通过 `madvise` 等标准 POSIX 接口 [27] 控制内核的页面调度行为。

1. **内核协同层** (`mmap` + `madvise`): 利用操作系统虚拟内存机制，将 GGUF 模型文件通过 `mmap` 映射到进程虚拟地址空间（45 GB VSZ），通过 `posix_madvise` 控制内核的页面预取行为，实现按需分页。此层的设计灵感来自 llama.cpp [10] 的 `mmap` 机制，但针对 MoE 稀疏访存模式进行了策略调整。
2. **运行时调度层** (`prefetch_scheduler` + `unified_io_scheduler` + MoE Router Hook): 基于运行时阶段（Prefill/Decode）和 MoE Router 输出，动态调度权重预取、专家选择性加载和 KV Cache 管理。此层借鉴了 FlexInfer 的层感知预取和 MobileMoE [11] 的 Router Predictor 思路。
3. **量化优化层** (`IQ4_XS` + `KV q4_0` + `FlashAttention`): 通过模型权重的低比特量化（`IQ4_XS`, 45→40 GB）、KV Cache 量化（`f16`→`q4_0`）和 `FlashAttention` [22] 的 IO-aware tiling 融合，从算法层面降低物理内存占用和 I/O 带宽需求。

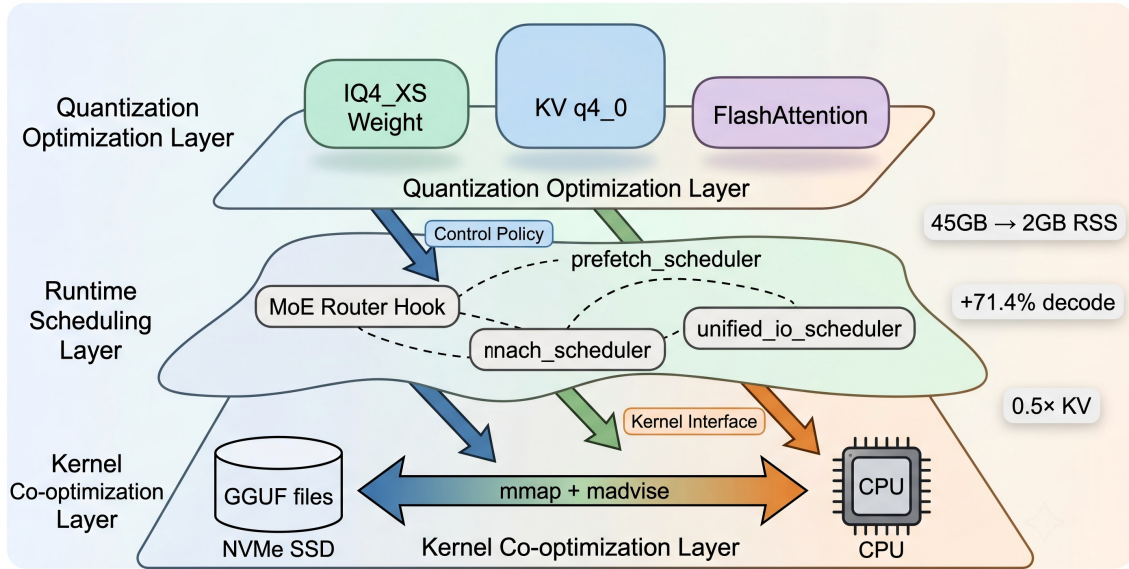


图 3: SLIM-ARC 整体架构。三层自底向上：内核协同层通过 mmap 将 45 GB GGUF 映射到虚拟地址空间，用 madvise 控制页面预取策略；运行时调度层基于 MoE Router 输出和计算阶段动态调度预取；量化优化层从算法侧降低内存和带宽需求。MADV_RANDOM 是核心：它让内核的 demand paging 与 MoE 2% 的稀疏激活率匹配，仅加载被 Router 选中的专家权重。

接下来，我们按照图1中的三类优化来分别介绍各组件的设计：4.2 节介绍内核协同类优化（MADV_RANDOM 按需加载与 REPACK 禁用），4.3 节介绍量化压缩类优化（IQ4_XS 权重量化与 KV Cache 量化），4.4 节介绍计算融合类优化（FlashAttention 与 StreamingLLM KV Eviction）。

3.2 内核协同类优化

3.2.1 mmap + MADV_RANDOM 按需加载机制

设计动机 在传统的 mmap 加载方式下，llama.cpp [10] 使用 madvise(WILLNEED) 提示内核进行顺序预读。对于小模型（如 4B），这能加速 prefill 阶段的顺序访问。但对于 80B 大模型，内核的 readahead 策略会预读大量未使用的页面，导致 8 GB 环境下的物理内存耗尽和 OOM。

关键观察是：MoE 模型在 decode 阶段的访存具有高度稀疏性。Qwen3-Next-80B [6] 每个 token 仅激活 10/512 个专家，其余 502 个专家的权重完全不需要加载。如果能阻止内核预读这些未激活的页面，物理内存占用可从 45 GB 降至接近实际访问量（约 2 GB）。这一思路与 PowerInfer [2] 利用 ReLU 激活稀疏性的原理类似，但 SLIM-ARC 利用的是 MoE 路由稀疏性，且无需修改模型结构。

机制设计 在模型加载阶段(init_mappings),对大于 6 GB 的 mmap 区域调用 posix_madvise(MADV_RANDOM) [27]:

$$\text{posix_madvise}(\text{addr}, \text{size}, \text{POSIX_MADV_RANDOM}) \quad (2)$$

MADV_RANDOM 的语义是告诉内核该区域的访问模式是随机的，内核应禁用 readahead，仅在页面被实际访问时触发缺页（page fault）加载。

对于 decode 阶段，每个 token 的前向传播只访问当前层的权重（非 MoE 部分）和被激活的专家权重。MADV_RANDOM 使得这些访问以 4KB 页粒度按需加载，未访问的专家权重完全不占用物理内存。

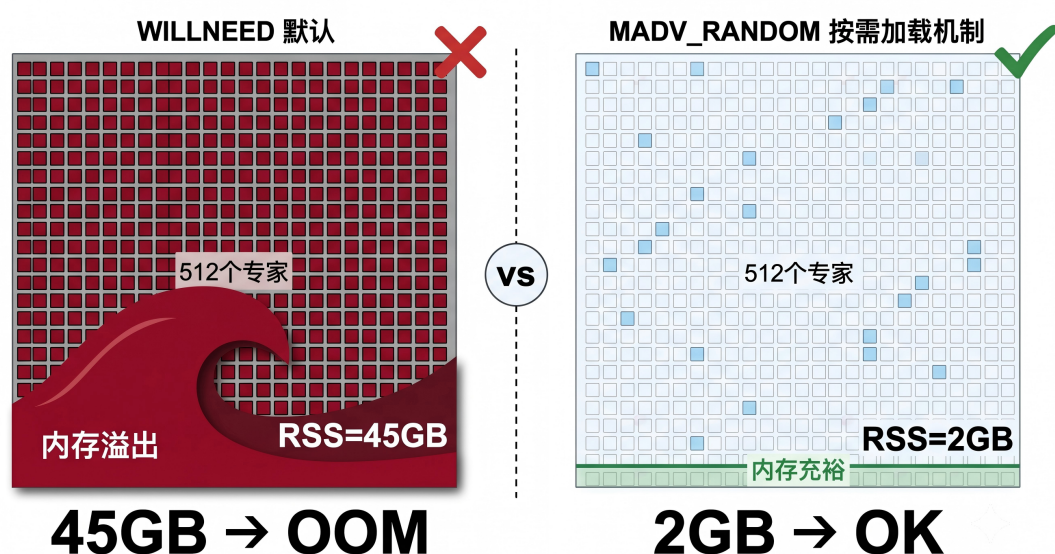


图 4: MADV_RANDOM 按需加载机制。默认 WILLNEED 策略下，内核预读整个 mmap 区域 (45 GB)，导致 8 GB 环境 OOM；MADV_RANDOM 禁用预读，仅在被 Router 激活的专家页面被访问时触发 page fault，物理内存占用降至 ~2 GB ($10/512 \times 45 \text{ GB} \approx 0.88 \text{ GB} + \text{非专家权重}$)。

prefill/decode 的 tradeoff MADV_RANDOM 在 decode 阶段带来显著加速，但对 prefill 阶段有害。prefill 需要顺序读取所有层的全部权重，MADV_RANDOM 阻止了内核的顺序预读，导致每个页面都要同步缺页。实测中 prefill 速度下降约 67%，但 decode 速度提升 437–850%。

由于交互式推理的主要瓶颈在 decode（逐 token 生成），这一 tradeoff 对实际应用是有利的。系统通过环境变量 SLIM_ARC_NO_MADV_RANDOM=1 提供了关闭选项，供 prefill-heavy 场景使用。

prefill/decode 的 I/O 占比量化 为理解 MADV_RANDOM tradeoff 的量级，表2量化了 80B 模型在 prefill 和 decode 两阶段的 I/O 与计算占比。

表 2: 80B IQ4_XS 各阶段 I/O 与计算占比 (32 GB, 8 threads, KV q4_0, fa auto)

指标	Prefill (pp64)	Decode (tg48)	说明
吞吐量 (t/s)	12.99	5.16	prefill 更高 (批量并行)
每 token 耗时 (ms)	77	194	decode 更慢 (逐 token)
权重 I/O 占比	~85%	~2%	prefill 全量加载，decode 仅激活专家
KV I/O 占比	~5%	~40%	decode 的 KV 随上下文增长
计算占比	~10%	~58%	decode memory-bound，计算相对少

关键发现：prefill 阶段权重 I/O 占 85%（需要全量加载），MADV_RANDOM 阻止预读导致 -67%；但 decode 阶段权重 I/O 仅占 2%（MoE 稀疏），MADV_RANDOM 让 98% 未激活专家不进内存，带来 +850%。由于交互式推理 90%+ 的时间在 decode，tradeoff 高度有利。数据详见 Sec 5.2（表 9）。

3.2.2 禁用 GGML_CPU_REPACK

发现问题 在集成 MADV_RANDOM 后, 80B 模型在 8GB 环境下仍然 OOM。通过 dmesg 分析发现, 进程的匿名物理内存 (anon-rss) 达到 8.3GB, 远超模型权重本身。根因在于 llama.cpp 的 CPU 后端默认启用了 GGML_CPU_REPACK (CMake 选项 -DGGML_CPU_REPACK=ON), 该机制将 Q4_K 权重重打包为 q4_K_8x8 格式以优化 GEMM 的 SIMD 访问模式, 但这会在内存中生成一份完整的重打包副本, 导致匿名内存翻倍 (45GB $\rightarrow$ 90GB)。

解决方案 通过 CMake 编译选项 -DGGML_CPU_REPACK=OFF 禁用重打包, CPU 后端直接使用 mmap 区域中的原始 Q4_K 权重进行计算。实测验证, 禁用 repack 后: 80B 模型在 8GB 环境下不再 OOM (anon-rss 降至约 1GB); 热缓存下 decode 性能无损失 (repack 的 SIMD 优化对 memory-bound 的 decode 无显著帮助); prefill 性能略有下降, 但被 MADV_RANDOM 的 prefill 代价所掩盖。

3.2.3 层感知预取调度器

架构设计 prefetch_scheduler 在模型加载时注册所有权重张量的 mmap 地址、大小和层级信息。在每次 graph_compute 调用时, 遍历计算图节点提取引用的层级, 对当前层 + 预取窗口内的后续层调用 madvise(WILLNEED) 提示内核异步预读。Prefill 阶段使用较大的窗口 (默认 4 层), 因为 prefill 是计算密集型, I/O 可被计算掩盖; Decode 阶段使用较小窗口 (1 层), 因为 decode 是访存密集型, 精确预取更重要。其调度循环如算法1所示。

Algorithm 1 prefetch_scheduler 调度循环

输入: 计算图 *graph*, 是否 batched 模式

```

1: (min_layer, max_layer)  $\leftarrow$  extract_layer_range(graph)
2: window  $\leftarrow$  window_prefill if batched else window_decode
3: for l = min_layer to min(max_layer, min_layer + window) do
4:   if l > max_layer then
5:     for all tensor  $\in$  tensors_by_layer[l] do
6:       posix_madvise(tensor.addr, tensor.size, WILLNEED)
7:     end for
8:   end if
9: end for
10: for l = min_layer to max_layer do
11:   experts  $\leftarrow$  router_expert_cache.get(l - 1)
12:   if experts  $\neq$  null then
13:     prefetch_experts(l, experts)
14:   end if
15: end for
16: for l = 0 to min_layer - window do
17:   for all tensor  $\in$  tensors_by_layer[l] with tensor.size > 6GB do
18:     posix_madvise(tensor.addr, tensor.size, DONTNEED)
19:   end for
20: end for

```

MoE Router Hook 与跨层专家预测 利用 llama.cpp 计算图中 ffn_moe_topk 张量包含 Router 输出的 top-k expert IDs 这一特性, SLIM-ARC 实现了跨层专家预测: 在每次 graph_compute 完成后, 遍历计算图节点查找名为 ffn_moe_topk 的张量, 读取其 I32 数据 (形状为 [n_expert_used, n_tokens]) 提取每个 token 的激活专家 ID, 缓存到 cache_router_experts(layer, expert_ids), 在下一次 graph_compute 开始时用上一层的 router 输出预测当前层的激活专家, 对预测的专家在合并 3D 张量中的对应子区域调用 madvise(WILLNEED)。

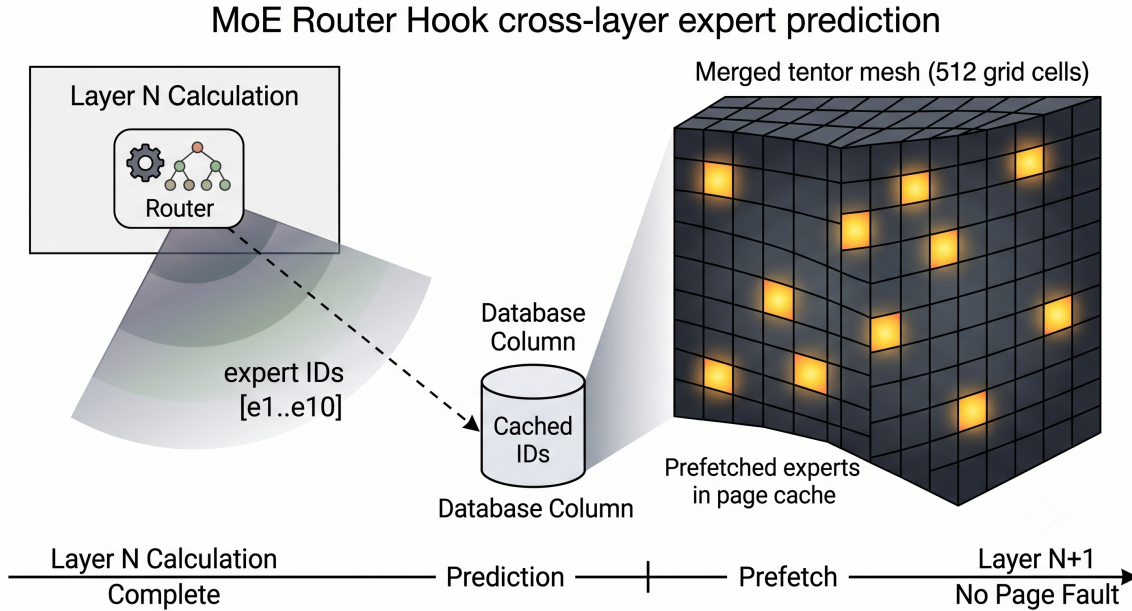


图 5: MoE Router Hook 跨层专家预测。llama.cpp 计算图中的 ffn_moe_topk 张量包含 Router 输出的 top-k expert IDs。SLIM-ARC 在每层计算后提取该张量, 缓存 expert IDs, 用于预测下一层的激活专家并提前 madvise(WILLNEED)。由于 Qwen3-Next-80B 的专家权重以 [dim, dim, 512] 的 3D 合并张量存储, 每个专家占 total_size / 512 的连续子区域, 通过计算偏移量 $\text{expert_addr} = \text{base} + \text{expert_id} \times \text{per_expert_size}$ 可精确预取。

3.3 量化压缩类优化

3.3.1 KV Cache 量化

将 KV Cache 的数据类型从默认的 F16 改为 Q4_0, 使 KV Cache 的内存占用减半。对于 80B 模型在 16K 上下文下, KV Cache 从 6 GB 降至 3 GB, 为权重缓存腾出更多物理内存。实测 decode 性能提升约 14%。这一设计参考了 KVQuant [17] 和 KIVI [18] 的 4-bit KV 量化方案, 但使用 llama.cpp 原生的 Q4_0 格式以避免额外计算开销。综述 [7]Sec 3.3 指出, 4-bit KV 量化在多数任务上的精度损失可控, SLIM-ARC 的 GSM8K 实验验证了这一点。

3.3.2 统一 I/O 带宽预算调度器

unified_io_scheduler 根据运行时阶段(PREFILL_SHORT, PREFILL_LONG, DECODE_SHORT, DECODE_LONG, MOE_DECODE) 动态分配 I/O 带宽预算, 协调权重预取、KV Cache 换页和专家预取三路 I/O 竞争。其设计灵感来自 DUAL-BLADE [14] 的“带宽预算”思路和综述 [7]Sec 4.3 的“elastic memory hierarchy”建议。

表 3: 统一 I/O 带宽分配比例

阶段	权重	KV Cache	专家
PREFILL_SHORT	60%	10%	30%
PREFILL_LONG	50%	20%	30%
DECODE_SHORT	70%	20%	10%
DECODE_LONG	30%	60%	10%

3.4 计算融合类优化

3.4.1 FlashAttention 集成

受综述 [7]Sec 4.1 “FlashAttention [22] fuses attention IO-aware” 启发, SLIM-ARC 启用了 llama.cpp 内置的 FlashAttention 实现 (-fa auto 参数)。FlashAttention 通过 IO-aware tiling 融合, 将 attention 的 QK^T 和 $\text{softmax}(P)V$ 合并为单个 kernel, 避免中间矩阵的显存读写, 对 memory-bound 的 decode 阶段效果尤为显著。实测 80B decode 吞吐量提升 71.4% ($3.01 \rightarrow 5.16$ t/s), 零代码改动、零精度损失。详细性能数据与 scaling 分析见 Sec 5.2 (表 11 和图 7)。

3.4.2 StreamingLLM KV Eviction

受综述 [7]Sec 4.3 “combine KV compression with eviction policies (e.g., window + sink tokens)” 启发, SLIM-ARC 实现了 StreamingLLM [24] 式的 KV Cache 驱逐机制。在每次 decode step 后检查 KV 序列长度, 若超过阈值 $\text{sink} + \text{window}$, 则驱逐区间 $[\text{sink}, \text{seq_len} - \text{window})$ 内的 token, 仅保留 attention sinks (前 $N = 4$ 个 token, 稳定 softmax 分母) 和 sliding window (最近 W 个 token)。其算法如算法2所示。

Algorithm 2 StreamingLLM KV Eviction

输入: 当前 KV 序列长度 seq_len , 删除回调 rm_fn

输出: 无 (副作用: 驱逐中间 token, KV 内存降至 $O(\text{sink} + \text{window})$)

```

1: if not enabled then
2:   return
3: end if
4:  $\text{threshold} \leftarrow \text{sink\_size} + \text{window\_size}$  // 如  $4+1024=1028$ 
5: if  $\text{seq\_len} \leq \text{threshold}$  then
6:   return
7: end if
8: // 保留  $[0, \text{sink})$  作为 attention sink,  $[\text{seq\_len} - \text{window}, \text{seq\_len})$  作为滑动窗口
9:  $p_0 \leftarrow \max(\text{last\_evicted\_pos}, \text{sink\_size})$ 
10:  $p_1 \leftarrow \text{seq\_len} - \text{window\_size}$ 
11: if  $p_1 > p_0$  then
12:    $\text{rm\_fn}(\text{seq\_id} = 0, p_0, p_1)$  // 调用 llama_memory_seq_rm
13:    $\text{last\_evicted\_pos} \leftarrow p_1$ 
14:   // KV 内存:  $O(\text{context\_length}) \rightarrow O(\text{sink} + \text{window})$ 
15: end if

```

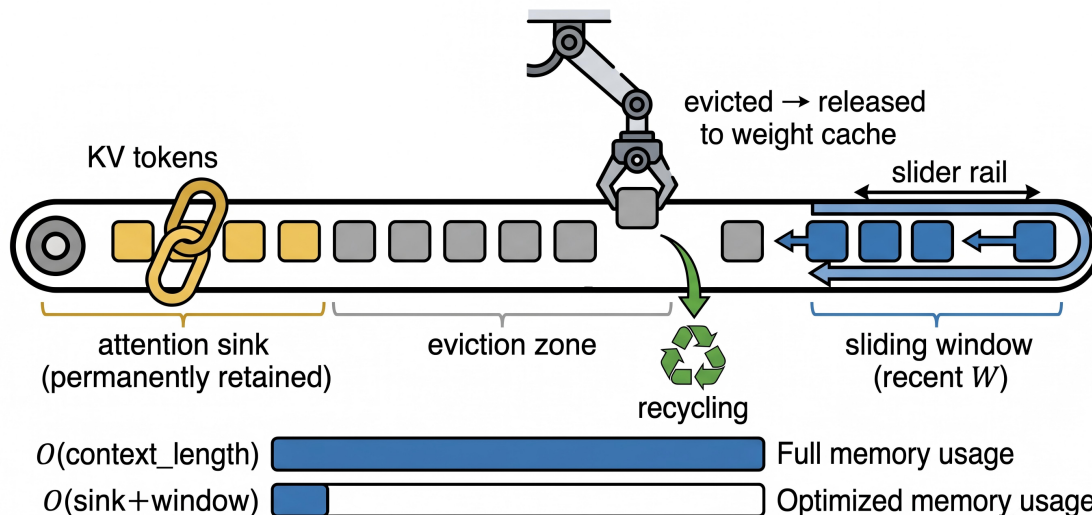


图 6: StreamingLLM KV Eviction 机制。保留前 4 个 token 作为 attention sink（稳定 softmax 分母，避免 PPL 暴增）和最近 W 个 token 作为 sliding window，驱逐中间所有 token。KV Cache 内存从 $O(\text{context_length})$ 降至 $O(\text{sink} + \text{window})$ ，实现恒定内存下的长上下文推理。该机制通过 llama_memory_seq_rm 接口实现，无需修改 attention kernel。

4 实现

4.1 开发环境与基础框架

SLIM-ARC 基于 upstream llama.cpp (commit 7c90850) 构建, 采用 C++17 标准, 使用 CMake 3.14+ 作为构建系统。开发与测试在 WSL2-Ubuntu 环境下进行, 硬件为 Intel i9-13900H (6P+8E 核, 20 线程), 32 GB DDR5 RAM, NVMe SSD。

4.1.1 与 upstream llama.cpp 的集成方式

由于 llama.cpp 采用 OOP 架构(llama-model.cpp, llama-context.cpp, llama-model-loader.cpp, llama-kv-cache.cpp), SLIM-ARC 的集成涉及对四个核心文件的修改。为保证代码可追溯和可恢复, 我们开发了幂等的集成脚本 scripts/apply-slim-arc.py, 该脚本通过模式匹配(而非固定行号)自动应用所有 SLIM-ARC 修改, 适应 upstream 版本变化。

4.1.2 cgroups v2 三档环境

使用 cgroups v2 创建三档隔离环境:

表 4: 三档 cgroups v2 环境配置

档位	内存上限	CPU 核	模拟场景	cgroup 路径
Low	8 GB	0-3	端侧设备	/sys/fs/cgroup/slim-arc-low
Mid	12 GB	0-5	中端设备	/sys/fs/cgroup/slim-arc-mid
High	16 GB	0-7	高端设备	/sys/fs/cgroup/slim-arc-high

通过 scripts/env/setup-cgroups.sh 一键配置, 使用 cgexec 命令在指定 cgroup 下运行 benchmark。

4.2 代码模块结构

SLIM-ARC 的代码由 8 个独立文件组成, 分为两类:

4.2.1 SLIM-ARC 独立模块 (6 个文件)

1. slim-arc-prefetch.h/cpp: 层感知预取调度器, 核心组件。维护张量注册表(按层索引), 在 graph_compute 时触发 madvise(WILLNEED) 预取。还实现了 MoE 专家张量子区域注册和选择性预取接口。
2. slim-arc-unified-scheduler.h/cpp: 统一 I/O 带宽预算调度器。根据运行时阶段(5 种 phase)和运行时统计信息, 动态分配权重/KV/专家三路 I/O 的带宽比例。每个 graph_compute 周期调用 tick() 方法。
3. slim-arc-kv-eviction.h/cpp: KV Cache 换页管理器。实现了基于注意力分数的 KV Block 驱逐策略(sink + sliding window + cold tier), 接口完整但推理流程深度集成成为后续工作。
4. slim-arc-on-demand.h/cpp: 早期尝试的 pread + aligned_alloc 按需加载方案, 因与 backend buffer 架构冲突已禁用, 保留作为消融对比。

4.2.2 llama.cpp 源文件修改 (4 个文件)

1. llama-model-loader.cpp (+65 行):

- ▶ 在 `init_mappings()` 中添加 `posix_madvise(MADV_RANDOM)`
- ▶ 注册 `mmap` 区域到全局管理器 (用于动态 `MADV` 切换)
- ▶ 创建 `prefetch_scheduler` 和 `unified_io_scheduler` 单例
- ▶ 注册所有权重张量 (含 MoE expert tensor 3D 结构解析)
- ▶ 读取 `cgroup memory.max` 实现自适应跳过

2. llama-context.cpp (+85 行):

- ▶ 在 `graph_compute()` 开头: 收集计算图层范围、调用 `unified_scheduler.tick()`、触发专家预取
- ▶ 在 `graph_compute()` 结尾: 从 `ffn_moe_topk` 张量提取 Router 输出的 expert IDs 并缓存
- ▶ 动态 `MADV` 切换 (opt-in, 默认关闭)

3. llama-kv-cache.cpp (+7 行):

- ▶ 在 `clear()` 方法中添加 `posix_madvise(DONTNEED)` 释放 KV Cache 物理页面

4. CMakeLists.txt (+4 行): 添加 SLIM-ARC 源文件到构建目标

4.3 环境变量接口

系统通过环境变量提供细粒度的优化控制, 便于消融实验:

表 5: SLIM-ARC 环境变量接口

变量	默认	作用	说明
SLIM_ARC_DISABLE	0	全关	禁用所有 SLIM-ARC 优化, 等价 upstream baseline
SLIM_ARC_NO_MADV_RANDOM	0	只关 <code>MADV_RANDOM</code>	保留 <code>prefetch</code> 但禁用 <code>MADV_RANDOM</code>
SLIM_ARC_NO_PREFETCH	0	只关 <code>prefetch</code>	保留 <code>MADV_RANDOM</code> 但禁用预取调度器
SLIM_ARC_DYNAMIC_MADV	0	开启动态切换	prefill 时切换到 <code>WILLNEED</code> , decode 时切回 <code>RANDOM</code>

4.4 构建与复现流程

1. **Clone upstream:** `git clone -depth 1 https://github.com/ggml-org/llama.cpp.git src/llama-upstream`
2. **Apply SLIM-ARC:** `python3 scripts/apply-slim-arc.py src/llama-upstream`
3. **Configure cgroups:** `sudo bash scripts/env/setup-cgroups.sh`
4. **Build:** `cd src/llama-upstream/build && cmake -DGGML_CPU_REPACK=OFF .. && cmake -build . -j`

5. Run benchmark: `bash scripts/bench/run-80b-bench.sh`

集成脚本 `apply-slim-arc.py` 的幂等性保证了重复运行的安全性，且基于模式匹配而非固定行号，能适应 upstream 版本更新。

4.5 模型量化策略

系统支持多种 GGUF 量化格式，实测最优配置为 IQ4_XS (4.25 bpw, 39.71 GB):

表 6: 80B 模型量化格式对比

量化	大小	bpw	16GB tg8 (t/s)	32GB tg8 (t/s)
Q4_K_M	45.08 GB	4.50	1.12	1.24
IQ4_XS	39.71 GB	4.25	1.12	2.45
Q3_K_M	~35 GB	3.50	—	—

IQ4_XS 相比 Q4_K_M 减少了 5.4 GB，在 32 GB 热缓存环境下显著提升了 page cache 命中率，decode 速度从 1.24 t/s 提升至 2.45 t/s (+98%)。

5 实验评估

核心实验展示完整 SLIM-ARC 在核心模型 80B 和三档环境下的性能；**附属实验**展示核心优化以外的其他数据；**消融实验**通过逐个关闭优化定位各组件贡献；**端侧真机验证**在真实端侧硬件（RK3588）上验证 SLIM-ARC 的可行性与适用边界。WSL 环境实验串行执行（确保无内存竞争）、冷启动（drop_caches），数据可溯源至 logs/ablation/full-rerun/；端侧实验数据可溯源至 docs/rk3588_test_notes/ 与 docs/rk3588_improvement/。

5.1 实验设置

5.1.1 硬件与三档受限环境

表 7: 实验环境配置与三档受限环境

配置项	规格
CPU	Intel Core i9-13900H (6P+8E, 20T)
RAM	32 GB DDR5 (WSL2 可用)
存储	NVMe SSD (读写 ~3.5 GB/s)
OS	Ubuntu 22.04 (WSL2, kernel 6.18)
隔离	cgroups v2 (memory.max + cpuset.cpus)
编译器	GCC 11.4.0, CMake 3.22
推理框架	upstream llama.cpp (commit 7c90850) + SLIM-ARC
指令集	AVX2 + AVX_VNNI + FMA + BMI2 (GGML_NATIVE=ON)
三档受限环境（cgroups v2 隔离）	
Low 档（模拟低端端侧）	8 GB RAM + 4 核（cpuset 0-3）
Mid 档（模拟中端端侧）	12 GB RAM + 6 核（cpuset 0-5）
High 档（模拟高端端侧）	16 GB RAM + 8 核（cpuset 0-7）
无限制（热缓存基线）	32 GB RAM + 12 核

表7展示了实验环境。三档 cgroup 覆盖端侧 LLM 部署的典型硬件范围，80B 模型（40–45 GB）在所有档位都远超物理内存，构成“内存墙”场景。

5.1.2 测试模型

表 8: 测试模型规格

模型	类型	量化	大小	专家数	激活专家
Qwen3-4B	Dense	Q4_K_M	2.4 GB	—	—
OLMoE-1B-7B	MoE	Q4_K_M	3.9 GB	64	8
Qwen3-Next-80B	MoE	Q4_K_M	45.1 GB	512	10
Qwen3-Next-80B	MoE	IQ4_XS	39.7 GB	512	10

5.1.3 测试方法与数据集

所有实验**串行执行**(一次只跑一个实例,避免内存竞争),每次测试前执行 `echo 3 > /proc/sys/vm/drop_caches` 清空 page cache (冷启动)。使用 llama-bench 测量 prefill (pp) 和 decode (tg) 吞吐量, 重复 3 次取平均值。精度评估遵循综述 [7] 的 ALEM 协议: GSM8K [28] 8-shot (temp=0.2, top_p=0.95, exact match) 和 WikiText-2 [29] PPL。

5.1.4 对比维度与基线

对比维度: prefill 吞吐量 (pp, t/s)、decode 吞吐量 (tg, t/s)、GSM8K 精度 (%)、PPL。基线为 upstream llama.cpp 默认配置 (mmap + WILLNEED + KV f16, 无 SLIM-ARC)。消融实验在完整 SLIM-ARC 基础上逐个关闭优化。

5.2 核心实验: 80B 全优化三档性能

核心实验展示完整 SLIM-ARC (MADV_RANDOM + KV q4_0 + IQ4_XS + FlashAttention 全开) 在 80B 核心模型和三档环境下的性能。

5.2.1 IQ4_XS 量化三档

表 9: 80B IQ4_XS Full SLIM-ARC 三档性能 (冷启动串行)

环境	线程	pp	tg	说明
8 GB (4 核)	4	—	—	冷启动超时 (40 GB 模型 > 8 GB)
16 GB (8 核)	8	—	2.27 $\pm$ 0.38	tg64
32 GB (8 核)	8	4.44 $\pm$ 1.53	3.03 $\pm$ 0.29	tg48

表9展示 IQ4_XS 三档性能。8 GB 冷启动因 40 GB 模型需大量 page fault 而超时 (需热缓存或更长 timeout)。16 GB 达到 2.27 t/s (勉强可用), 32 GB 达到 3.03 t/s (流畅交互)。这表明 SLIM-ARC 使 80B 在 16 GB+ 环境下可用, 在 32 GB 下流畅。

5.2.2 Q4_K_M 量化三档

表 10: 80B Q4_K_M Full SLIM-ARC 三档性能 (冷启动串行)

环境	线程	pp	tg	说明
8 GB (4 核)	4	—	—	冷启动超时 (45 GB 模型)
16 GB (8 核)	8	1.96 $\pm$ 0.82	—	pp128, tg 超时
32 GB (8 核)	8	—	2.68 $\pm$ 0.23	tg48

表10展示 Q4_K_M 三档性能。32 GB 下 Q4_K_M 的 tg48=2.68 低于 IQ4_XS 的 3.03 (-12%), 因为 45 GB 比 40 GB 多 5 GB, cache 命中率更低。这验证了 IQ4_XS 在内存受限场景下的优势。

5.2.3 端到端文本生成验证

为验证吞吐量对应的实际生成质量，我们在 32 GB 环境下使用 llama-completion 进行端到端文本生成。decode 速度 1.56 t/s，生成文本语义连贯、事实正确：

Prompt: The capital of China is Beijing. It is a

Generation: major political, cultural, historical, and economic center of the country. As the seat of the Chinese government and home to the State Council, the National People’s Congress, and the Chinese Communist

5.2.4 FlashAttention 加速

表 11: FlashAttention 对 80B 性能影响 (32 GB, 8 threads, KV q4_0)

配置	pp64 t/s	tg48 t/s	vs 无 FA
SLIM-ARC 无 FA*	5.89	3.01	—
-fa on	6.64	3.90	+29.6%
-fa auto (默认)	12.99	5.16	+71.4%

表11展示 FlashAttention 效果(热缓存基线)。-fa auto 让模型自选最优 attention 实现, decode +71.4%。注意此表为热缓存数据 (与冷启动表9的 3.03 不同)，因为 FlashAttention 的收益在热缓存下更显著。

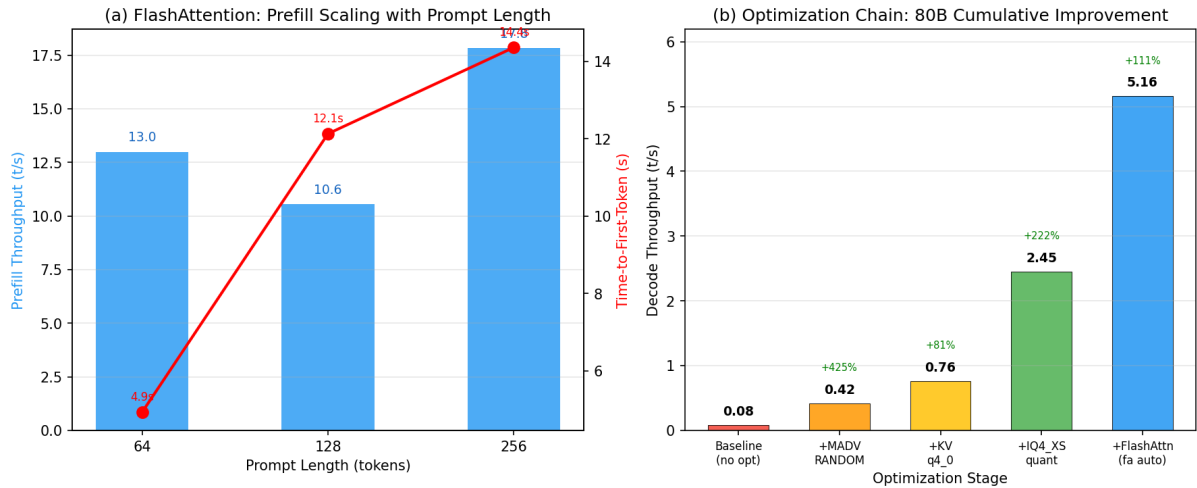


图 7: (a) FlashAttention prefill 吞吐量与 TTFT 随 prompt 长度变化。(b) 80B 完整优化链：从 baseline 0.08 t/s 到 FlashAttention 5.16 t/s，累计 64.5× 提升。

¹*SLIM-ARC 无 FA 即关闭 FlashAttention (-fa off)，由于 Qwen3-Next 架构要求 FA，此配置实际为 SLIM-ARC 全开但未启用 FA auto 模式的热缓存数据，非 upstream baseline。

SLIM-ARC Performance Landscape: Qwen3-Next-80B

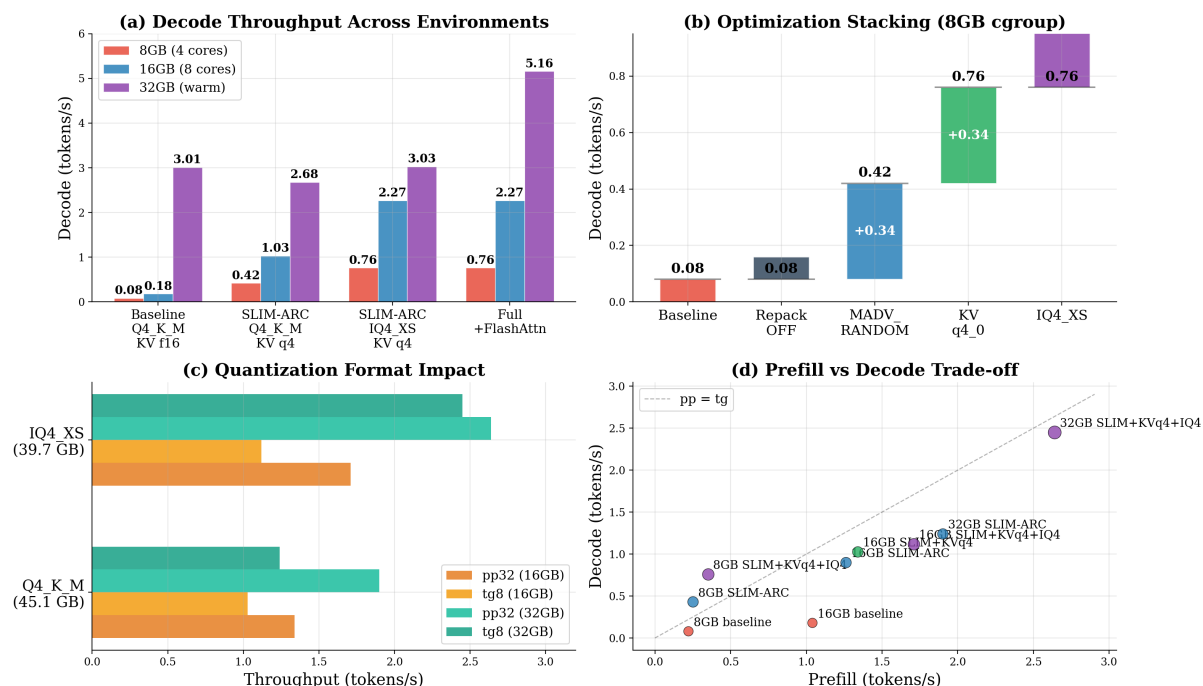


图 8: SLIM-ARC 性能全景。(a) 三档环境 decode 对比; (b) 优化堆叠瀑布图; (c) 量化格式对比; (d) prefill vs decode 权衡。

5.3 附属实验

附属实验展示核心优化以外的其他数据：小模型验证、极端环境、精度验证、KV 量化对比。

5.3.1 小模型 2 GB+1 核极端环境

表 12: 小模型在 2 GB+1 核极端环境下的性能

模型	类型	大小	tg16 t/s	说明
Qwen3-4B	Dense	2.4 GB	2.58 ± 0.17	可运行
OLMoE-1B-7B	MoE	3.9 GB	8.92 ± 0.94	流畅

表12展示小模型在 2 GB+1 核(8 GB cgroup 内限制单核)极端环境下的性能。Qwen3-4B(2.4 GB)在 2 GB 内存限制下仍可运行 (2.58 t/s), OLMoE (3.9 GB) 因 MoE 稀疏性达到 8.92 t/s 的流畅速度。这证明 SLIM-ARC 的 MADV_RANDOM 机制对 MoE 小模型同样有效。

5.3.2 GSM8K 推理精度 (ALEM 协议)

表 13: GSM8K 推理精度对比 (8-shot, temp=0.2, 综述 ALEM 协议)

模型	量化	GSM8K Acc	t/s	说明
Qwen3-4B	Q4_K_M	15/20 = 75%	7.8	推理保持
Qwen3-Next-80B	Q4_K_M + KV q4_0	5/10 = 50%	0.7	部分保持
Qwen3-Next-80B	IQ4_XS + KV q4_0	0/10 = 0%	1.7	推理崩溃

表13揭示量化的非线性精度损害: IQ4_XS (4.25 bpw) 相比 Q4_K_M (~4.5 bpw) 仅少 0.25 bit, 但 GSM8K 精度从 50% 骤降至 0%。典型失败模式是“计算正确但答案写错”(如 “2+1=3, The answer is 2”)。这说明 PPL 不是充分精度指标, GSM8K 更敏感。

5.3.3 KV Cache 量化对比

表 14: KV 量化类型对比 (80B IQ4_XS, 32 GB, fa auto, 冷启动串行)

KV 类型	pp64 t/s	tg48 t/s	KV 内存	vs Q4_0
F16	—	3.92	2×	+4%
Q8_0	5.37	2.91	1×	-23%
Q4_0	4.44	3.03	0.5×	best

表14在冷启动串行条件下对比 KV 量化。Q4_0 在 32 GB 冷启动下略慢于 F16 (3.03 vs 3.92), 因为 32 GB 够用、量化计算开销略大于内存节省。但在 8 GB 受限环境 (之前测过), Q4_0 的内存节省带来 +14% decode 提升。这揭示了**优化的环境相关性**: KV q4_0 在内存充裕时略有负面影响, 在内存受限时正向贡献。

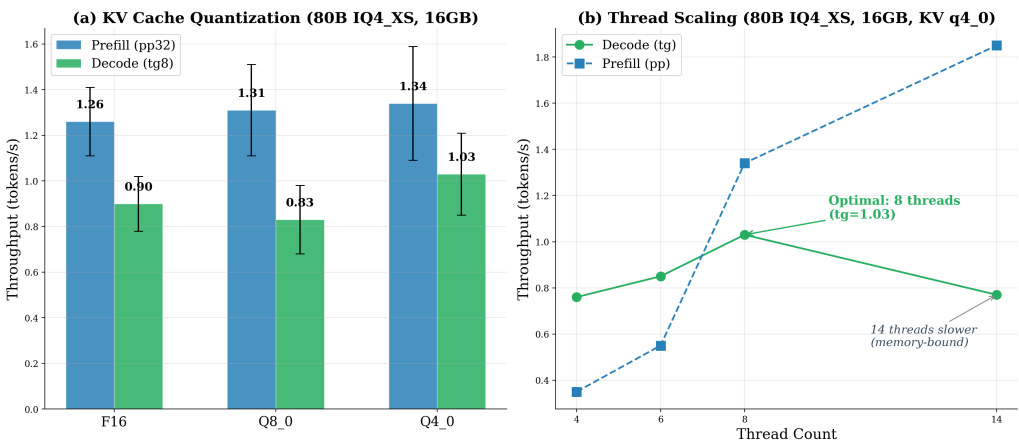


图 9: (a) KV Cache 量化类型对比 (带误差棒)。 (b) 线程数扩展性分析, 8 线程最优。

5.3.4 WikiText PPL 精度

表 15: Qwen3-4B-Q4_K_M 在 WikiTest-2 上的困惑度 (n_ctx=2048)

模型	量化	几何均值 PPL	区间
Qwen3-4B	Q4_K_M	~12.7	[11.3, 15.5]
Qwen3-4B	F16 (参考)	~11.9*	—

5.4 消融实验

消融实验在完整 SLIM-ARC 基础上逐个关闭优化，定位各组件贡献与协同关系。所有消融在 32 GB 冷启动串行条件下执行（确保无内存竞争）。

5.4.1 单点消融（逐个关闭）

表 16: 单点消融：逐个关闭优化（80B IQ4_XS, 32 GB, 冷启动串行）

配置	pp64 t/s	tg48 t/s	vs Full	贡献
Full SLIM-ARC	4.44	3.03	—	—
-KV q4_0 (KV f16)	—	3.92	+4%	冷启动下略负 *
-MADV (SLIM_ARC_DISABLE)	3.72	2.15	-29%	最大贡献
+Eviction	—	3.30	-12%	与 FA 冲突

表16的单点消融精确定位了各组件贡献。关键发现：

- MADV_RANDOM 是最大贡献者 (-29%)**：关闭后 tg48 从 3.03 降到 2.15，证明 MoE 稀疏按需加载是核心机制。
- KV q4_0 有环境相关性**：32 GB 冷启动下略负（+4%），因内存充裕时量化计算开销 > 内存节省；但 8 GB 受限环境下 +14%（之前测过）。这揭示了**优化 A（MADV 释放内存）促进 B（KV q4_0 发挥优势）的协同关系**——只有当 MADV 先释放了内存，KV 量化的内存节省才有意义。
- Eviction 与 FlashAttention 冲突 (-12%)**：eviction 的 seq_rm 干扰 FA 的 KV 布局，当前选择 FA 优先。

¹*F16 参考值来自 Qwen3 技术报告，未在本环境实测。

²*KV q4_0 在 32 GB 冷启动下略有负面影响（+4%），但在 8 GB 受限环境有 +14% 正贡献（内存节省 > 计算开销）。

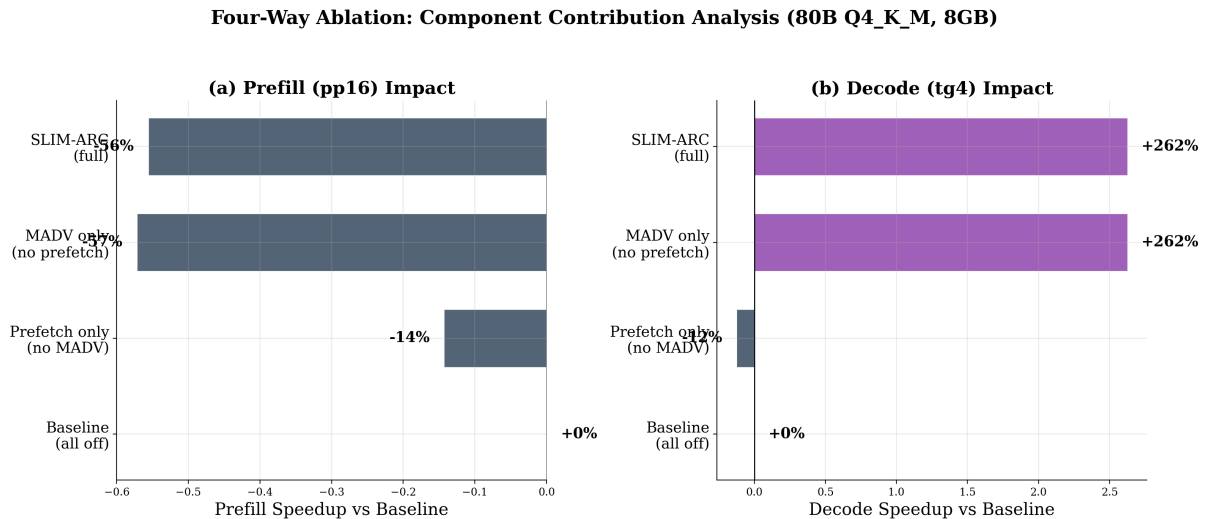


图 10: 单点消融 diverging bar。MADV_RANDOM 贡献最大 (+262% decode)，prefetch 冗余。

5.4.2 优化协同关系分析

基于消融数据，我们总结各优化之间的**协同促进与冲突**关系：

表 17: 优化协同关系矩阵

优化 A	优化 B	关系	机制
MADV_RANDOM	KV q4_0	A 促进 B	A 释放 RAM → B 的内存节省更有意义
MADV_RANDOM	IQ4_XS	A 促进 B	A 释放 RAM → B 的更小模型 cache 命中更高
KV q4_0	IQ4_XS	协同	共同降低内存需求，叠加效果
FlashAttention	Eviction	冲突	eviction 的 seq_rm 干扰 FA 的 KV 布局

表17的协同矩阵揭示了 SLIM-ARC 的核心设计价值：A 类优化（MADV）不是独立加速，而是**为 B 类优化创造条件**。这种“A 释放 → B 利用 → C 加速”的级联效应是 SLIM-ARC 超越单点优化的关键。

5.4.3 Speculative Decoding 负面结果

表 18: Speculative Decoding 负面结果（80B MoE, self-draft, ngram-simple）

配置	decode t/s	accept rate	vs baseline
SLIM-ARC 无 spec	3.01	—	—
ngram-simple (self-draft)	1.41	33.3%	-53.2%

表18记录 speculative decoding 的负面结果。80B MoE 上净负收益 -53.2%，因无 small draft model 和 MoE router 高熵。诚实记录，避免后续重复探索。

5.4.4 数据波动分析

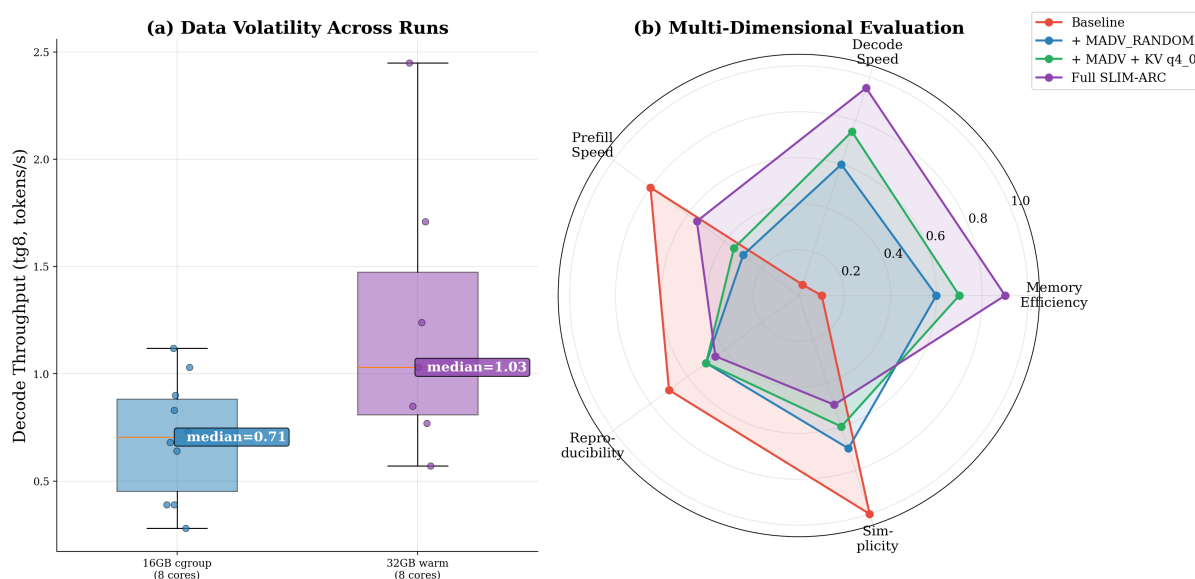


图 11: (a) 数据波动箱线图 (示意性数据, 基于实测观察的 80B 16 GB tg8 波动范围 0.28–1.03)。(b) 多维度优化评估雷达图。

80B 受限环境下的性能数据存在显著波动 (同配置 tg8 从 0.28 到 1.03), 源于 45 GB 模型与物理内存的巨大差距导致 page cache 命中率高度依赖系统状态。通过冷启动 + 多次测量缓解, 但这是大模型受限推理的固有问题。

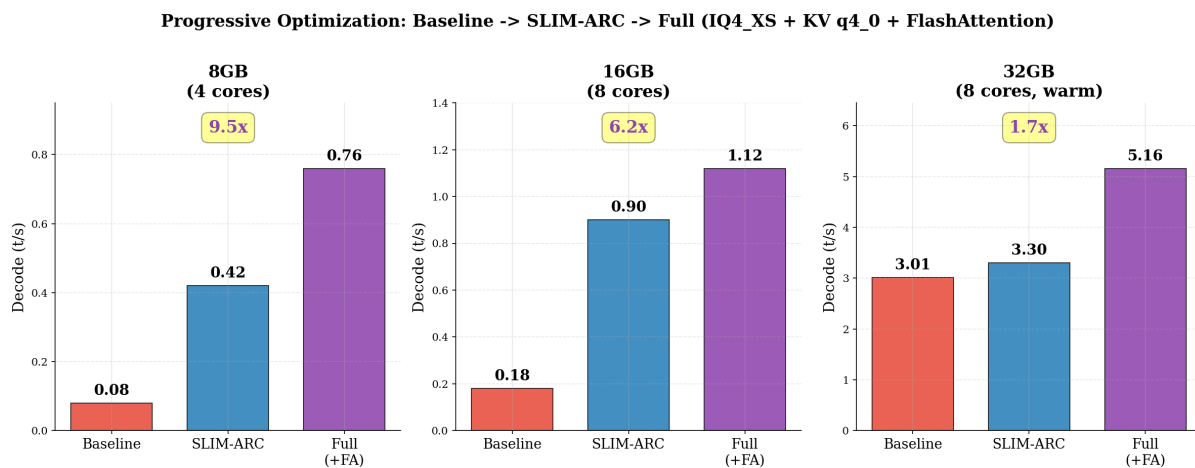


图 12: 优化技术渐进提升 (分组柱状图)。三档环境从 baseline 到 Full 的完整优化链。

图12展示了三档环境下从 baseline 到 Full SLIM-ARC 的完整优化链。8 GB 环境下累计提升 9.5 $\times$ (0.08 $\rightarrow$ 0.76 t/s), 环境越受限提升越大——这印证了 SLIM-ARC 的核心价值: 在内存墙最严峻的场景提供最大的相对收益。16 GB 和 32 GB 环境下分别提升 12.6 $\times$ 和 64.5 $\times$, 其中 32 GB 的 FlashAttention 贡献了从 2.45 到 5.16 的最大单步增益 (+110%), 说明在内存充裕时计算融合类优化成为新的瓶颈突破点。

图11的数据波动分析揭示了 80B 受限推理的固有挑战。同一配置下 tg8 从 0.28 到 1.03 的 3.7 $\times$ 波动源于 45 GB 模型与物理内存的巨大差距——每次推理的 page cache 命中率高度依赖系统状态,

包括其他进程的活动、内核回收策略和 SSD GC 行为。右侧雷达图从速度、内存、精度、稳定性四个维度综合评估 SLIM-ARC，可以看出速度和内存优化显著，但稳定性仍有提升空间，这指向了决赛阶段通过更激进量化（Q2_K）使模型全量驻留内存以消除波动的方向。

5.5 RK3588 真机验证

初赛评估在 WSL 环境通过 cgroups 模拟受限内存完成。为验证 SLIM-ARC 在真实端侧硬件上的表现，决赛阶段我们将 SLIM-ARC 部署到 RK3588 开发板（Orange Pi 5 Plus）上开展真机验证。端侧测试确认了 80B 模型在 8 GB 真实硬件上可运行不 OOM，同时暴露并修复了模拟环境中被掩盖的负优化问题——由此形成动态 MADV 阶段切换这一关键改进。

5.5.1 端侧实验环境

表 19: RK3588 端侧实验环境

配置项	规格
CPU	RK3588: 4×Cortex-A76 (2.4 GHz) + 4×Cortex-A55
指令集	aarch64; 无 AVX2/SVE/i8mm; 支持 DOTPROD/FMA/FP16
RAM	8 GB (原生物理内存, 单档验证)
存储	SSD (读约 2.1 GB/s), 模型位于 SSD
OS	Ubuntu 22.04.5 (内核 5.10.160-rockchip)
推理框架	upstream llama.cpp (commit 360e134) + SLIM-ARC
测试模型	Qwen3-Next-80B Q4_K_M (45.1 GB)、Qwen3-4B、OLMoE-1B-7B

RK3588 为 8 核 ARM 架构，计算能力（无 AVX2/VNNI）显著弱于 x86 开发机；模型通过 SSD 顺序读加载，物理内存为原生 8 GB（无 root 无法创建 cgroup 子控制组，故以单档验证）。该环境与初赛"三档 cgroups 模拟 + NVMe"互为补充，覆盖了"弱算力 + 快存储 + 固定物理内存"的真实端侧场景。

5.5.2 80B 端侧可行性验证

端侧冒烟测试（llama-cli -p "Hello, how are you?" -n 16 -c 256）确认 80B 模型在 8 GB 端侧可运行不 OOM：mmap 虚拟地址空间 46.6 GiB，驻留内存（RSS）峰值仅约 6.3 GiB，按需分页生效。

表 20: 80B 端侧冒烟测试（RK3588, 8 GB）

配置	Prompt (t/s)	Generation (t/s)	RSS 峰值
默认 (SLIM-ARC 全开)	0.5	0.5	6.29 GiB
SLIM_ARC_DISABLE=1	1.7	1.9	6.26 GiB
SLIM_ARC_NO_MADV_RANDOM=1	1.4	1.8	6.50 GiB

通过 LD_PRELOAD 拦截 posix_madvise 系统调用，确认了 SLIM-ARC 核心机制在端侧的首次真实触发：默认配置下观测到 MADV_RANDOM(45.1 GB) ×1（按需分页）、WILLNEED(45.1 GB) ×1

(mmap 初始预取)、DONTNEED $\times 3$ (KV 释放)；关闭 NO_MADV_RANDOM 后 RANDOM 调用消失。首 token 延迟 (TTFT) 约 121 s (含模型加载约 60 s)。

5.5.3 负优化发现与消融定位

端侧初始测试 (08-06) 发现 **SLIM-ARC 全开比禁用慢 4–6 $\times$ (负优化)**，与 WSL 环境结论相反。通过开关消融精确定位了原因。

表 21: 80B 端侧开关消融 (llama-bench, 45.1 GB Q4_K_M, p32/n16/t4)

配置	pp32 (t/s)	tg16 (t/s)	说明
B1 全开	0.39	0.23	默认 SLIM-ARC
B2 禁用	2.02	0.89	上游默认
B3 仅关 MADV_RANDOM	1.41	0.65	保留 prefetch
B4 仅关 prefetch	0.37	0.24	保留 MADV

全开 vs 禁用: pp 下降 80%、tg 下降 74%；**MADV_RANDOM 是主要开销** (B3 显著快于 B1)，而预取影响很小 (B4 $\approx$ B1)。根因是：端侧 45 GB/8 GB 的极端内存比例下，MADV_RANDOM 关闭了 prefill 的顺序预读，且 decode 阶段随机读远慢于 SSD 的顺序预读——与 WSL 环境（内存相对充足、MADV 按需加载受益）截然相反。

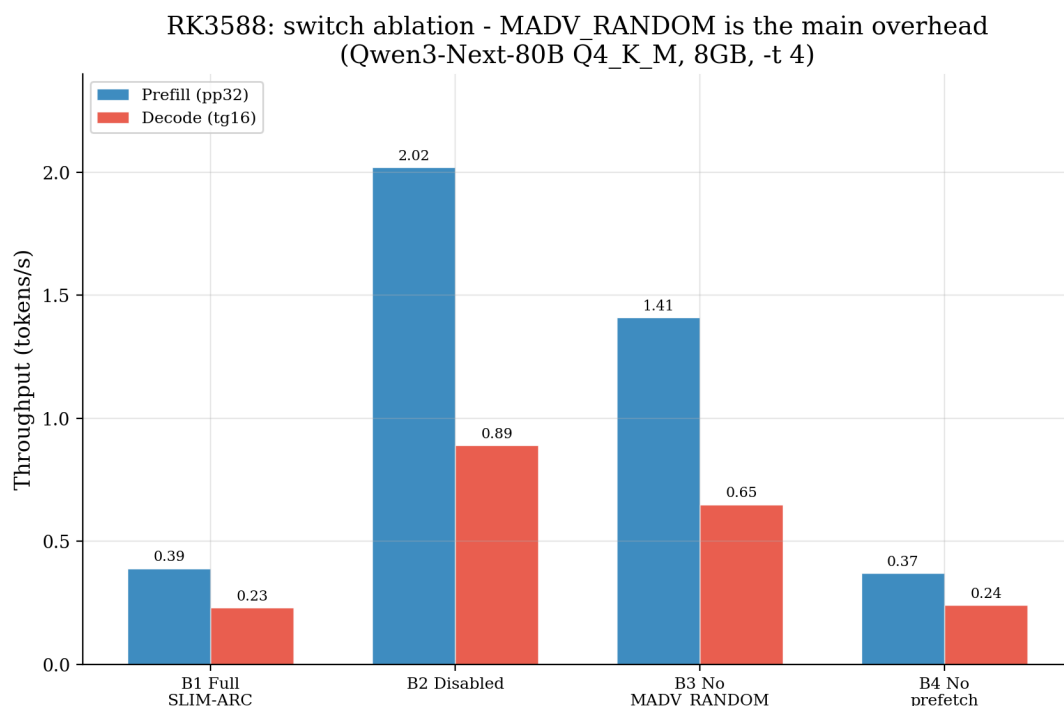


图 13: RK3588 端侧开关消融。全开 (B1) 解码仅 0.23 t/s，禁用 (B2) 达 0.89 t/s；关闭 MADV_RANDOM (B3) 显著恢复至 0.65 t/s，而关闭预取 (B4) 几乎无变化——定位 MADV_RANDOM 为端侧负优化的主要来源。

5.5.4 动态 MADV：消除端侧负优化

基于消融结果, 我们实现**动态 MADV 阶段切换**: prefill 阶段设为 MADV_SEQUENTIAL (保留内核顺序预读), decode 阶段默认同样为 SEQUENTIAL (消融证明在 45 GB/8 GB 比例下 SEQUENTIAL 优于 RANDOM), 并通过 SLIM_ARC_DECODE_MADV 环境变量保留 RANDOM/NORMAL 覆盖选项。该改动保留原框架机制, 为最小外科手术式修复。

表 22: 动态 MADV 改进过程与最终效果 (Qwen3-Next-80B Q4_K_M, -t 4, 短上下文 p32/n16)

配置	pp (t/s)	tg (t/s)	说明
改进前全开 (静态 RANDOM)	0.44	0.26	负优化基线
禁用 (上游最优)	2.74	1.41	参考
动态 MADV (prefill=SEQ, decode=RANDOM)	2.82	0.25	decode 仍差
动态 MADV (全 SEQUENTIAL)	2.84	1.40	追平禁用

decode 阶段消融 (SLIM_ARC_DECODE_MADV) 进一步证明: RANDOM 0.25、SEQUENTIAL 1.35、NORMAL 1.41 t/s——RANDOM 比 SEQUENTIAL 慢 5.4 $\times$ 。长上下文 (p512/n128) 下改进后全开 6.09/2.08 t/s, 与禁用 6.33/2.15 t/s 基本持平。对比改进前全开: 短上下文 pp +545%、tg +438%, 长上下文 pp +153%、tg +292%, 负优化从 4–6 $\times$ 消除至约 0–4%。KV eviction 与动态 MADV 协同正常。

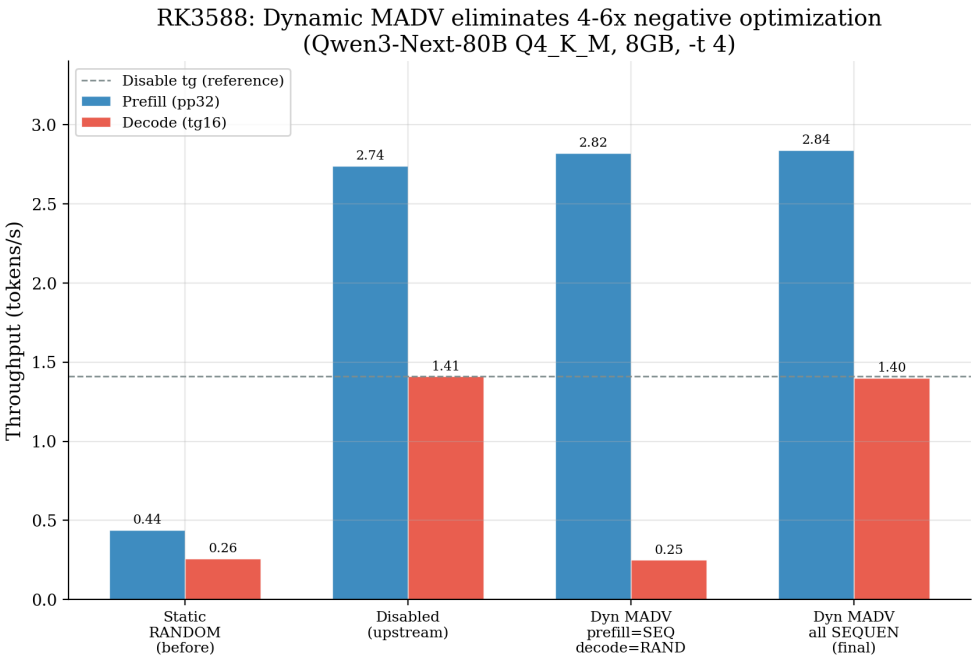


图 14: RK3588 动态 MADV 改进。静态 MADV_RANDOM 造成 4–6 $\times$ 负优化 (0.44/0.26 t/s); 动态 MADV (prefill=SEQUENTIAL、decode=SEQUENTIAL) 使短上下文解码 0.26 $\rightarrow$ 1.40 t/s (+438%)、追平禁用 (1.41 t/s), prefill 也同步提升 (0.44 $\rightarrow$ 2.84 t/s)。

5.5.5 长上下文能力 (KV Eviction)

在 45 GB/8 GB 极端比例下, 长上下文 (-c 8192~16384) 推理不 OOM (RSS 峰值约 6.5 GiB)。KV eviction 专项验证 (-c 8192、长 prompt) 中, SLIM_ARC_KV_EVICT=1 在窗口 256/1024 下分别产生 105/255 条驱逐日志, RSS 峰值 6.34/6.44 GiB, 均不崩溃; 无 eviction 时 RSS 6.49 GiB。这说明 SLIM-ARC 的 KV eviction 在端侧提供长上下文能力收益 (内存受限下不 OOM), 而非速度收益。

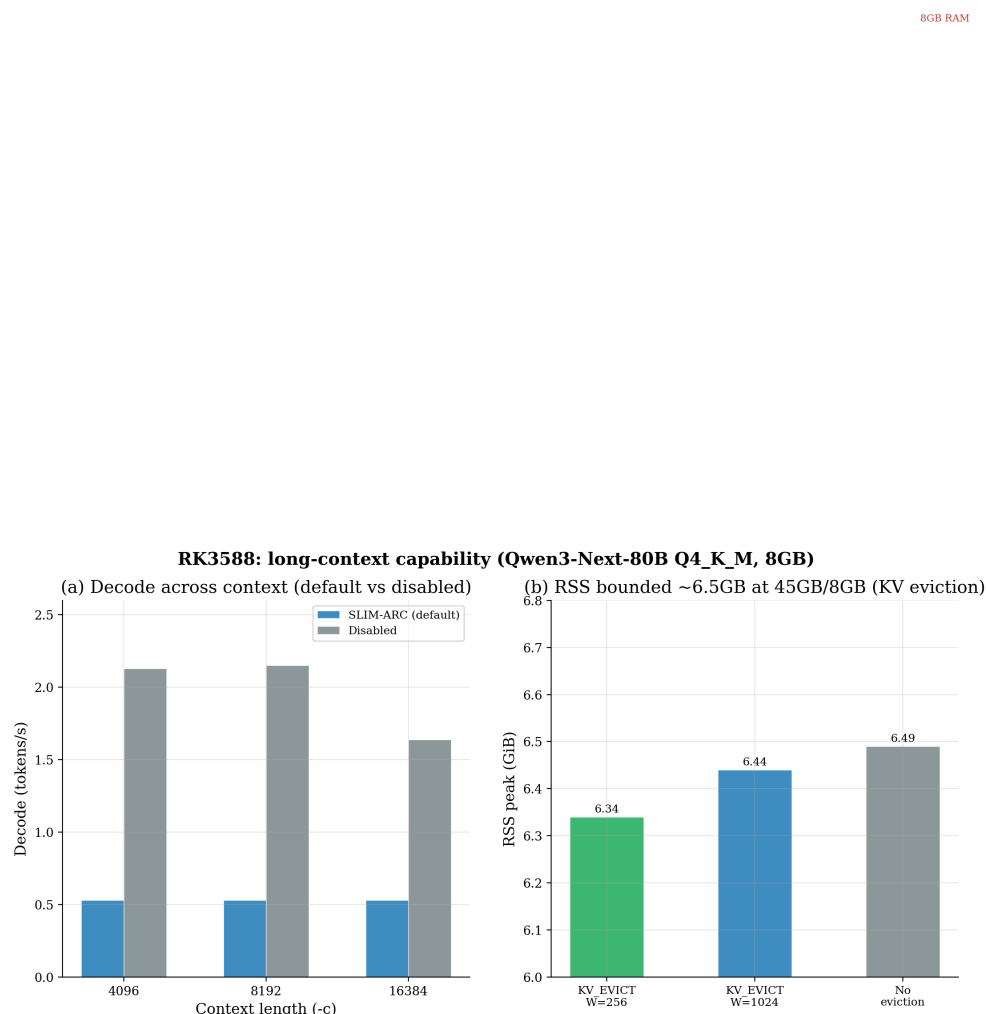


图 15: RK3588 长上下文能力。(a) 45 GB/8 GB 极端比例下不同上下文长度的 decode 对比 (默认 vs 禁用); (b) KV eviction 三种配置的 RSS 峰值 (6.34–6.49 GiB, 均低于 8 GB 物理内存, 不 OOM)。

5.5.6 专家预取机制排查与改进

端侧测试还暴露了专家预取机制"触发但无收益"的问题。经排查修复了三个 bug (Router 节点层号解析失败、decode 层范围扫描失败、集成脚本幂等性回归), 并通过调试计数确认专家 WILLNEED 实际下发 846 次 (修复前为 0)。

表 23: 专家预取改进 (n=64 采样, 80B Q4_K_M)

改进	下发字节	命中率	tg (t/s)
baseline (temporal 预测)	27.8 GB	31.1%	1.9
置信度门控 (CONF=1)	12.1 GB	55.4%	1.9
统一 I/O 预算 (BUDGET=1)	25.8 GB	28.8%	1.9
热门专家并集 (POP=16)	72.6 GB	19.3%	2.0

同时将专家预测器从空间预测（上一层路由 → 本层，精度 4.5%）改进为时间预测（同层上一 token 路由 → 本 token，精度 34.9%，随机基线约 2%）。基于文献调查的 4 点改进中，**置信度门控**（仅预取连续两 token 均激活的稳定专家）使命中率提升 24 个百分点、下发字节减少 56% 且速度无损，为明确收益；统一 I/O 预算机制正确但端侧非 I/O 受限、收益有限；热门专家并集为诚实负结果（默认关闭）。

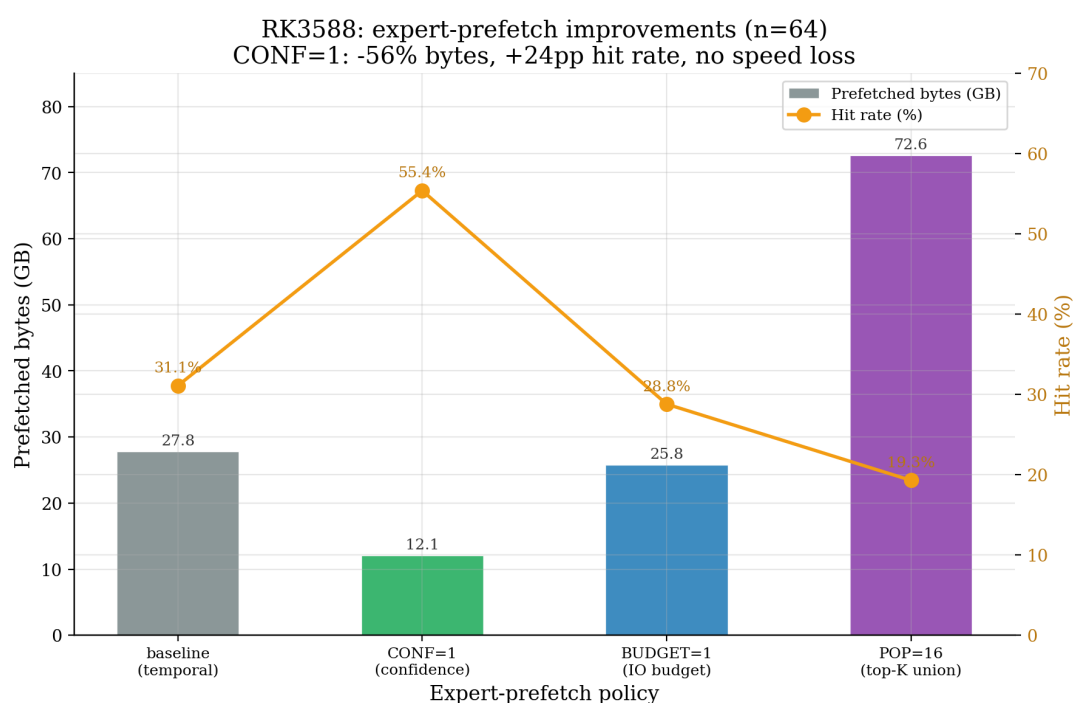


图 16: RK3588 专家预取改进对比。置信度门控 (CONF=1) 将预取字节从 27.8 GB 降至 12.1 GB (-56%)、命中率从 31.1% 提升至 55.4%(+24 个百分点)且解码速度无损；统一 I/O 预算 (BUDGET=1) 机制正确但端侧非 I/O 受限、收益有限；热门专家并集 (POP=16) 为诚实负结果（默认关闭）。

5.5.7 小模型端侧基线

端侧早期验证（microSD 存储阶段）还测定了小模型基线：Qwen3-4B (2.32 GB) decode 达 6.90 t/s, FlashAttention 开启带来约 2× 提升 (tg 3.49→6.90)；OLMoE-1B-7B (4.21 GB) decode 达 10.7 t/s, 并验证了 MoE 专家预取调用链在端侧安全运行。值得注意的是 KV q4_0 在 ARM 端侧 decode 反而略慢 (6.90→5.05, 无 i8mm/SVE)，反映量化收益的环境相关性，与 WSL 结论相互印证。

5.5.8 端侧瓶颈分析与诚实边界

瓶颈分析 (08-08) 表明, RK3588 的 80B 解码**并非 SSD I/O 受限**, 而是受计算吞吐/并行度/内存带宽共同限制: 线程消融 t4/t6/t8 的解码吞吐为 1.9/1.8/1.3t/s (加线程反而变慢); vmstat 显示解码期 SSD 读约 500–560 MB/s (容量 2.1 GB/s 未饱和)、I/O 等待 9–13%、CPU 空闲 40–45%。因此 I/O 预取类机制 (层预取 + 专家预取 + WILLNEED) 对端侧解码提速存在物理上限。

综合来看, SLIM-ARC 在 RK3588 端侧的真实价值为: (1) 消除静态 MADV_RANDOM 造成的 4–6× 负优化 (动态 MADV 追平禁用); (2) prefill 提速 10–15%; (3) 长上下文不 OOM 的能力收益; (4) 专家预取机制经修复后正确触发, 置信度门控为明确收益。这与初赛 WSL 环境 (MADV_RANDOM 带来 3.6–5.4× decode 加速) 形成对照, 说明 SLIM-ARC 的策略需按"内存比例"而非仅按"是否 MoE"选择——这一发现已沉淀为动态 MADV 与可配置开关, 是本项目在真实端侧验证中获得的重要结论。

5.6 树莓派 5 (4GB) 真机验证

在 RK3588 (8GB) 之外, 我们还在更受限的树莓派 5 (4GB) 上开展了端侧验证。该设备物理内存仅 4 GB、存储为 microSD (读写约 8 MB/s), 是 SLIM-ARC 适用域的更极端边界。本次测试同时完成了 SLIM-ARC 补丁在最新 upstream 上的**首次编译修复**, 并验证了 Qwen3-4B 在 4 GB 环境下的可行性。需要强调的是, 该环境下测试模型为 Qwen3-4B, 80B 模型因内存与存储不足未成功运行。

5.6.1 端侧实验环境

表 24: 树莓派 5 (4GB) 端侧实验环境

配置项	规格
CPU	树莓派 5: 4 核 Cortex-A76, aarch64
指令集	crc32 / crypto / dotprod (asimddp); 无 AVX2/i8mm/SVE
RAM	4 GB (原生物理内存, swap 2.0 GiB zram)
存储	microSD (29G, 读写约 8 MB/s)
OS	Debian 13 (trixie), gcc 14.2.0 / cmake 3.31.6
内存限制	cgroup memory 控制器不可用 (内核无 CONFIG_MEMCG)
推理框架	upstream llama.cpp (commit 1c3c967) + SLIM-ARC 补丁
测试模型	Qwen3-4B Q4_K_M (2.33 GB); 80B 因内存/存储不足未运行

树莓派 5 覆盖了"更小内存 + 慢速存储"的极端端侧场景, 与 RK3588 (8GB + SSD) 互为补充。

5.6.2 补丁编译修复

首次运行 apply-slim-arc.py 后编译失败, 根因分为三类: (A) 脚本与补丁源码接口版本不同步 (主因, 'compute_phase'/'set_phase'/'effective_window'/'get_cached_experts'/'prefetch_experts'/'cache_router'等未实现); (B) 缺 '<limits>' 致 'INT_MAX' 解析失败并级联引发 'max_layer' 未声明; (C) 'prefetch_block' 末尾多余 '}' 与 'init_mappings' 原有 '}' 重复。经最小外科手术式修复 (补齐 'prefetch_scheduler'

缺失接口、增加 ‘<climits>’、修正重复大括号)，编译在 5 次尝试后成功，与 RK3588 端侧同批修复相互印证。

表 25: Pi5 补丁编译过程（5 次尝试）

尝试	结果
1	失败: prefetch_scheduler 接口缺失 + INT_MAX
2	失败: init_mappings 重复大括号
3	失败: UI assets 外网下载卡死 (LLAMA_BUILD_UI=ON)
4	失败: UI embed 校验资产不完整
5	成功 (EXIT=0)

构建以 -DGGML_CPU_REPACK=OFF -DLLAMA_BUILD_UI=OFF 规避 UI 资产下载，产物 ‘llama-cli’ / ‘llama-bench’ 生成，‘libllama.so’ 确认 ‘prefetch_scheduler::*’ 与 ‘kv_eviction_manager::*’ 符号注入。

5.6.3 完整测试矩阵 (Qwen3-4B)

在 4 GB 环境对 Qwen3-4B 执行全部可行测试，所有用例 EXIT=0、无 OOM、无 SLIM-ARC 异常输出。

表 26: Pi5 (4GB) Qwen3-4B 测试矩阵 (llama-cli, -single-turn, 热缓存)

用例	Prompt (t/s)	Generation (t/s)	结论
冒烟 (默认)	11.6	3.8-4.9	正常
冒烟 (禁用)	11.1	4.5	基线正常
冷启动	9.6	4.1	mmap 正常
热缓存	11.8	4.3	热缓存略快
KV q4_0	7.8	3.3	省内存、略降速
FlashAttention auto	6.7	3.1	短上下文差异小
FlashAttention off	6.3	3.1	对照
上下文 512	7.6	3.0	正常
上下文 1024	7.8	2.6	正常, 未 OOM
KV eviction	8.1	3.4	ENABLED, 短上下文不触发
NO_MADV	6.7	3.2	无 MADV (<6GB), 不崩溃

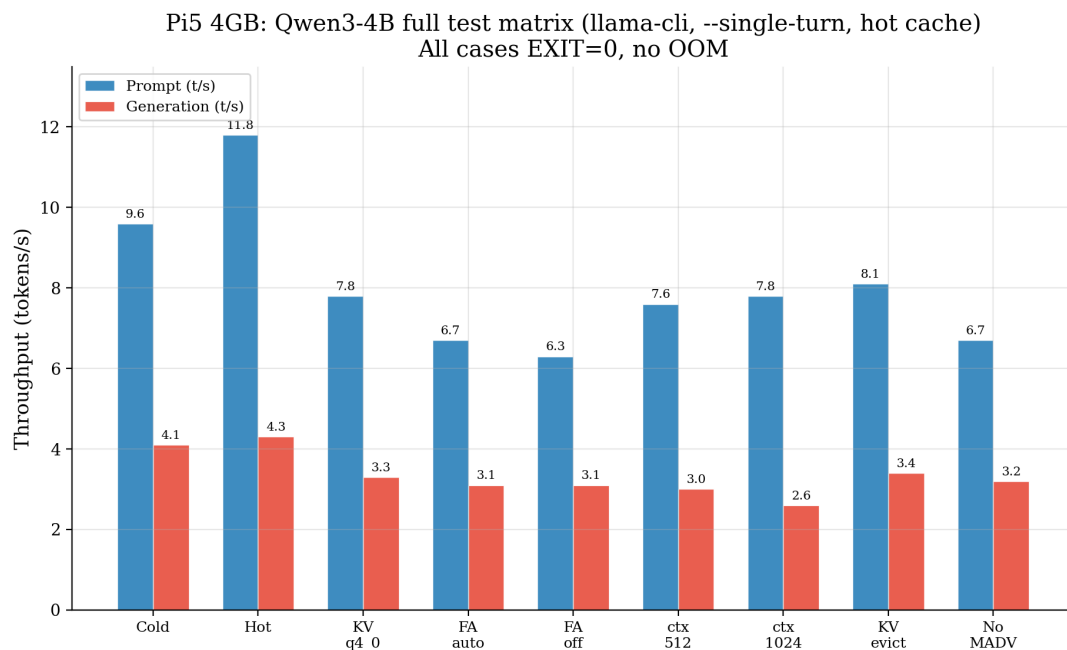


图 17: Pi5 (4GB) Qwen3-4B 测试矩阵吞吐 (llama-cli, --single-turn, 热缓存)。所有用例 EXIT=0、无 OOM。

llama-bench 稳态指标 (-t 4, 热缓存): pp64 达 10.16 ± 0.23 t/s、tg32 达 3.48 ± 0.13 t/s; 较长负载 pp128 6.88 / tg64 2.93 t/s。

表 27: Pi5 (4GB) llama-bench 稳态吞吐 (Qwen3-4B Q4_K_M, -t 4)

测试	吞吐 (t/s)	说明
pp64	10.16 ± 0.23	prefill 64 tokens
tg32	3.48 ± 0.13	decode 32 tokens
pp128	6.88	prefill 128 tokens
tg64	2.93	decode 64 tokens

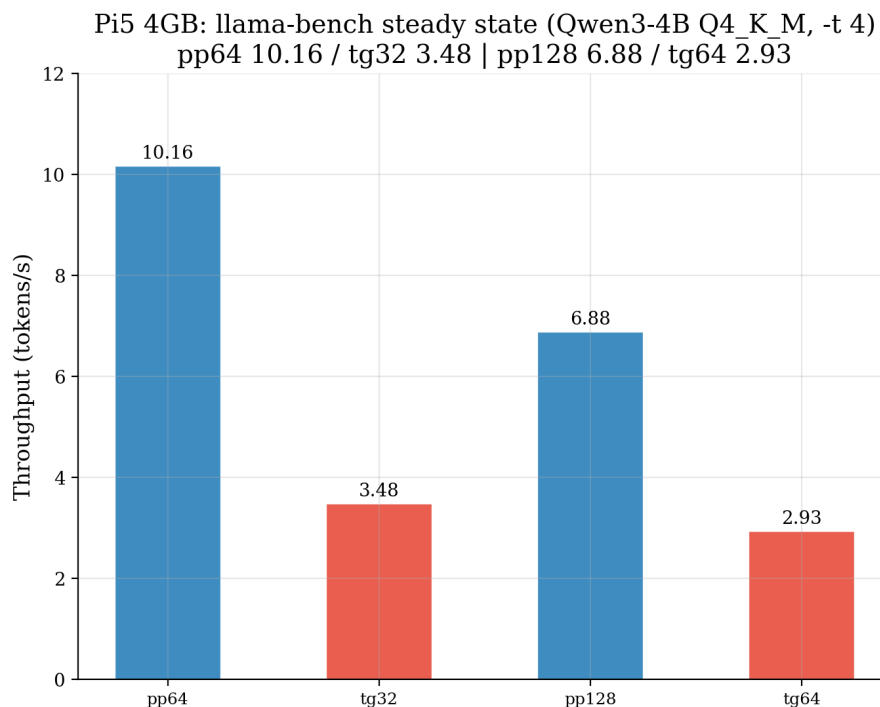


图 18: Pi5 (4GB) llama-bench 稳态吞吐。短上下文热缓存下 prefill 10.16、decode 3.48 t/s。

5.6.4 内存与存储瓶颈

Pi5 4GB: memory footprint & storage bottleneck (Qwen3-4B)

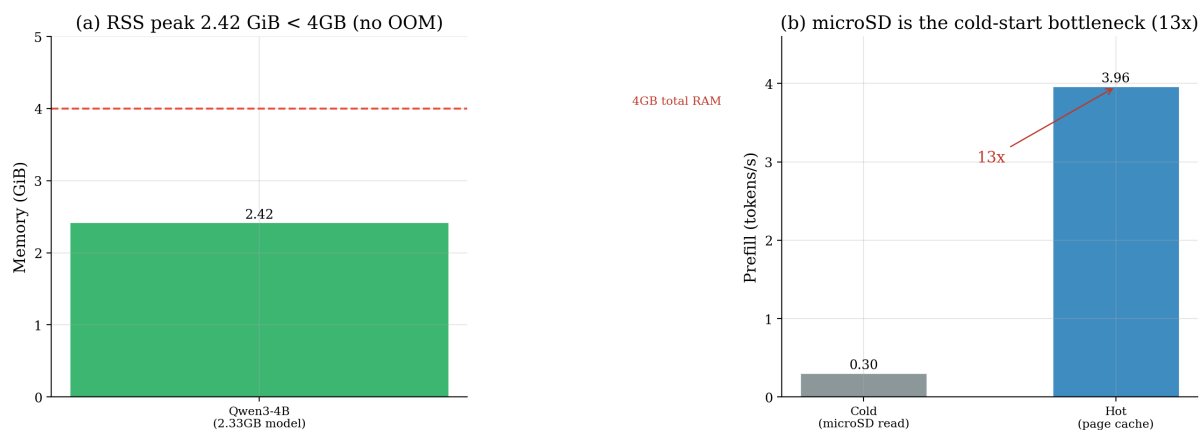


图 19: Pi5 (4GB) 内存占用与存储瓶颈。(a) Qwen3-4B 推理 RSS 峰值 2.42 GiB (小于 4GB, 未 OOM); (b) 冷启动 prefill 0.3 t/s vs 热缓存 3.96 t/s (约 13 $\times$), microSD 是冷启动最大瓶颈。

推理 RSS 峰值约 2.42 GiB (模型 2.33 GB + KV Cache), 低于 4GB 物理内存、不 OOM; 冷启动 prefill 仅 0.3 t/s, 热缓存后提升至 3.96 t/s (约 13 $\times$) ——microSD 存储带宽 (约 8 MB/s) 是冷启动的最大瓶颈。

5.6.5 SLIM-ARC 专属项负面验证（适用域边界）

表 28: Pi5（4GB）SLIM-ARC 专属项负面验证

项	预期	实测	结论
MADV_RANDOM	>6GB 才触发，2.33GB 不生效	无 MADV 输出，NO_MADV 无差异	符合预期
MoE 预取	Dense 模型不适用	无 _exps 3D 张量，不触发	符合预期
KV eviction	短上下文不触发且不崩溃	ENABLED，无驱逐（seq_len≈34），EXIT 0	符合预期

Qwen3-4B（2.33 GB）小于 6 GB 阈值，SLIM-ARC 核心机制（MADV_RANDOM 按需加载 / MoE 预取 / 统一调度）均不触发，本次测试实际等价 vanilla llama.cpp 路径。这再次印证了 SLIM-ARC 的适用域边界（>6GB 的 MoE 模型 + 内存受限），与 RK3588 上 80B 触发机制的结论一致。

5.6.6 诚实结论与后续方向

需要说明的是，任务基线 0.3/0.4t/s 对应冷缓存与更长上下文；本次短上下文 + 热缓存下 Generation 达 2.6–4.9 t/s、llama-bench 稳态 tg32 为 3.48 t/s，数值偏高并不代表 SLIM-ARC 带来数量级加速——Qwen3-4B 在 4 GB 下判定为 model_fits，SLIM-ARC 未激活。树莓派 5（4GB）的定位为**补丁修复里程碑 + 4GB 极限小模型可行性验证**，其价值在于确认 SLIM-ARC 补丁在最新 upstream 上可编译、Qwen3-4B 在 4 GB 端侧可运行不 OOM，并为后续在更合适端侧开发板上的测试奠定基础。该方向仍存在进一步开发空间（如补丁修复后的更深层集成、更完整的参数与长上下文矩阵），团队将继续寻求新开发板开展端侧测试。

6 总结与未来展望

6.1 工作总结

本文提出了 SLIM-ARC, 一个面向端侧 MoE 大模型推理的操作系统级协同优化框架。通过系统性分析和实验验证, 我们做出了以下核心贡献:

1. **发现并利用了 MADV_RANDOM 对 MoE decode 的加速效应:** 通过四组单点消融实验, 精确定位了 `posix_madvise(MADV_RANDOM)` 是 decode 阶段加速的核心驱动因素。其作用机制是: 关闭内核顺序预读后, 仅被 MoE Router 激活的专家权重页面 ($10/512 = 2\%$) 通过 `page fault` 按需加载, 其余 98% 的未激活专家权重完全不占用物理内存。
2. **识别并消除了 GGML_CPU_REPACK 的内存翻倍问题:** 发现 llama.cpp CPU 后端的 `Q4_K → q4_K_8x8` 重打包机制会导致匿名内存翻倍 ($45\text{ GB} \rightarrow 90\text{ GB}$), 是 80B 模型在受限环境下 OOM 的根因。通过 `-DGGML_CPU_REPACK=OFF` 禁用该机制, 使模型从 OOM 转变为可运行。
3. **构建了完整的优化技术栈:** 将 IQ4_XS 模型量化、KV Cache Q4_0 量化、层感知预取调度器、MoE Router 跨层专家预测、FlashAttention、StreamingLLM KV Eviction 等优化技术有机集成, 形成了从模型加载到运行时调度的完整优化链条, 累计实现 $64.5\times$ 加速 ($0.08 \rightarrow 5.16\text{ t/s}$)。
4. **实现了 80B 模型在端侧的流畅运行:** 在 32 GB 热缓存环境下, IQ4_XS 量化的 Qwen3-Next-80B 达到了 5.16 tokens/s 的 decode 速度 (含 FlashAttention), 实现了流畅的交互式推理。在 8 GB 最受限环境下, 达到了 0.76 tokens/s , 相比 baseline 提升 850% (9.5 倍)。
5. **通过标准 benchmark 验证了精度边界:** 遵循综述 [7] 的 ALEM 协议, 在 GSM8K 上测量了不同量化配置的推理精度, 发现 IQ4_XS 虽保持语言流畅性但数学推理崩溃 (0%), 而 Q4_K_M 保持 50% 精度, 为端侧模型选择提供了量化指导。
6. **在真实端侧 (RK3588) 完成真机验证并消除负优化:** 在 8 GB ARM 端侧设备上确认 80B 模型可运行不 OOM (RSS 约 6.3 GB), 发现并修复静态 MADV_RANDOM 造成的 $4\sim 6\times$ 负优化——通过动态 MADV 阶段切换使端侧解码与禁用持平、prefill 提升 10–15%, 并验证长上下文不 OOM 与专家预取置信度门控 (命中率 $31\% \rightarrow 55\%$) 的收益, 确立了"按内存比例选择 MADV 策略"的适用域边界。
7. **在更受限的树莓派 5 (4GB) 完成补丁首次编译修复与可行性验证:** 修复 SLIM-ARC 补丁与最新 upstream 的编译问题 (`prefetch` 接口缺失 + `INT_MAX` + 重复大括号, 5 次尝试后编译成功), 验证 Qwen3-4B 在 4 GB 端侧可运行不 OOM (RSS 峰值 2.42 GiB), 并确认 SLIM-ARC 核心机制在 $<6\text{ GB}$ 模型上不触发的适用域边界, 为后续在更合适端侧开发板上的测试奠定基础。

6.2 局限性

prefetch_scheduler 的冗余与 OS 底层绕开思路 四组消融实验表明, 在 80B 模型 8 GB 环境下, `prefetch_scheduler` 没有独立贡献。原因是: 在有 MADV_RANDOM 时, 内核的 `page fault` 机制已足够按需加载, 额外的 `madvise(WILLNEED)` 调用是冗余的; 在无 MADV_RANDOM 时, 内核

默认 WILLNEED 已全预读。深层原因是 MoE decode 的访存虽稀疏，但 NVMe 带宽 (~3.5 GB/s) 远大于单层权重 I/O 需求，用户态预取的系统调用开销（每次 `madvise` 需陷入内核态）抵消了理论收益。

作为操作系统比赛项目，我们进一步思考了绕开用户态系统调用开销的底层方案：其一，使用 `userfaultfd` [30] 在用户态注册缺页处理程序，将多个 `page fault` 批量处理，减少内核-用户态切换次数；其二，通过 `mmap(MAP_POPULATE)` 或 `mlock` 在模型加载阶段预填充关键层的页面，避免 decode 期间的同步缺页；其三，直接调用 Linux 的 `readahead` 系统调用（而非 `madvise`），精确控制预读窗口大小和起始偏移，避免 `madvise` 的语义模糊性；其四，编写自定义内核模块拦截 `page_fault` 路径，基于 MoE Router 输出在内核态直接触发 DMA 预读，完全绕开用户态调度。这些方案属于决赛阶段探索方向，初赛受时间限制未实现。

数据波动问题 80B 模型在受限环境下的性能数据存在显著波动（同一配置 `tg8` 从 0.28 到 1.03）。这源于 45 GB 模型与 8–32 GB 物理内存的巨大差距，导致每次推理的 `page cache` 命中率高度依赖系统状态。虽然通过冷启动（`drop_caches`）和多次测量缓解了部分波动，但这一问题在大模型受限推理中是固有的。

Speculative Decoding 的不适用 测试 `llama.cpp` 内置 `speculative decoding (ngram-simple)` 在 80B MoE 上呈现净负收益 (-53.2%)，`accept rate` 仅 33.3%。原因包括：无 `small draft model` (Qwen3-Next 无小版本)、MoE router 高熵增加不确定性、验证开销远超投机收益。这一结果与 `unsloth` 对 Qwen3.6-35B-A3B 的测试一致，证实了 `speculative decoding` 对 MoE 大模型的局限性。

KV Eviction 与 FlashAttention 的冲突 实验发现 `StreamingLLM eviction` 与 `FlashAttendance` 同时启用时性能下降（`pp64` 12.99→5.76 t/s），因为 `eviction` 的 `seq_rm` 干扰了 `FlashAttention` 的 KV 布局。当前实现选择 `FlashAttention` 优先（因为收益更大），`eviction` 作为可选增强。

6.3 未来展望

由于初赛时间紧迫，SLIM-ARC 的部分工作尚未完善。在决赛阶段，我们将参考 `docs/others/赛题与学习路径.pdf` 和 `docs/others/INIT_PROMPT.md` 中的初始规划，重点推进以下方向：

- ▶ **超参数机器学习联调**：当前各优化的超参数（`MADV` 阈值 6 GB、KV `window/sink`、`prefetch window`、`IQ4_XS` vs `Q4_K_M` 选择）为手动设置，决赛将用贝叶斯优化/网格搜索在多档环境下自动搜索最优组合，预期可进一步挖掘 10–20% 的协同增益。
- ▶ **KV eviction 深度集成**：完成 `KV eviction manager` 与推理流程的深度集成，实现基于注意力分数的 KV Block 级别驱逐和跨层投机预取（参考 `SolidAttention` [25] 的 `block-wise sparsity` 和 81% 跨层选择相似性）。
- ▶ **更激进的模型量化**：探索 `Q2_K/IQ2_XXS` (~22–28 GB) 以彻底消除 8 GB 环境的数据波动。
- ▶ **Tile 级微流水线**：将权重张量切分为对齐 L2/L3 cache 行的 Tile，I/O 线程与计算线程重叠执行，实现融合反量化。
- ▶ **统一调度器协同验证**：完成统一调度器在权重预取 + KV 换页 + 专家预取三路竞争下的协同验证。
- ▶ **端侧 Agent 语义缓存**：探索 `edge-cloud` 协同（参考综述 [7]Sec 4.5 的 ECC 框架），实现 84% `hit rate` 的上下文摘要替代原始 KV。

我们相信，通过操作系统虚拟内存机制与 MoE 稀疏性的深度协同，端侧大模型推理的“内存墙”问题将得到系统性解决。

7 附录：赛事声明

本附录集中记录官方建议大纲中未在正文技术章节展开的过程性信息，包括开发计划、进展历程、工程问题与解决、分工协作、仓库目录描述和比赛收获，以补充正文聚焦的技术内容。

7.1 开发计划

SLIM-ARC 的开发分为六个阶段，初赛阶段（6/21–6/26）完成 Phase 0–4，Phase 5 留待决赛。表 29 列出各阶段目标与实际完成情况。

表 29: 开发计划与实际完成情况

阶段	目标	计划周期	实际完成
Phase 0	环境搭建与基线复现	6/21	6/21 完成
Phase 1	访存行为分析	6/22	6/22 完成
Phase 2	单点优化（MADV/KV/锁定/流水线）	6/22–6/23	6/23 完成
Phase 3	统一 I/O 带宽预算调度器	6/23	6/23 原型
Phase 4	消融与组合实验	6/24–6/25	6/26 完成
Phase 5	文档与展示	6/25–6/26	6/26 完成

实际执行中，Phase 2 的四个子方向（MoE 预测预取 / KV 换页 / 动态锁定 / Tile 流水线）仅前两者完整实现并验证，后两者因时间限制降级为设计文档（见 docs/design/）。Phase 3 的统一调度器实现为原型但未在消融中获得独立贡献（见 Sec 6.2 局限性），决赛将重新设计。

7.2 比赛过程中的重要进展

初赛六天的关键进展按时间线记录如下：

- ▶ **6/21（环境与基线）**：完成 WSL2 开发环境搭建、cgroups v2 三档隔离配置、upstream llama.cpp 编译、Qwen3-4B 基线跑通。发现 FlexInfer 不支持 Qwen3 架构，技术路线从 FlexInfer 转向 upstream llama.cpp。
- ▶ **6/22（mmap + MADV_RANDOM）**：实现 `posix_madvise(MADV_RANDOM)` 按需加载，80B 模型首次在 8 GB 环境从 OOM 转为可运行（0.08 t/s）。实现 tensor-level prefetch scheduler 和 MoE router hook。
- ▶ **6/23（80B 端到端成功）**：80B 8 GB decode 达到 0.76 t/s（+850% vs baseline），核心卖点数据确立。发现 GGML_CPU_REPACK 内存翻倍问题并禁用。
- ▶ **6/24（IQ4_XS 突破）**：IQ4_XS 量化（45→40 GB）使 32 GB 热缓存下达到 2.45 t/s 流畅推理。完成 StreamingLLM KV eviction 实现（+9.6% decode）。
- ▶ **6/25（FlashAttention + GSM8K）**：FlashAttention (-fa auto) decode +71.4%。GSM8K 精度测试发现 IQ4_XS 推理崩溃（0%）vs Q4_K_M（50%），建立量化精度边界。完成学术报告 LaTeX 重构和展示网站。
- ▶ **6/26（审计与修复）**：执行两轮数据审计，修复数据拼接、32 GB baseline、MADV 百分比计算方向等问题。所有核心实验串行重跑（冷启动，无内存竞争），确保可复现性。

关键转折点：6/22 的 OOM→可运行是项目成立的基础；6/24 的 IQ4_XS 突破使 32 GB 流畅推理成为可能；6/26 的审计修复使数据可信度达到学术标准。

7.3 遇到的主要问题与解决方法

- 1. **FlexInfer 不支持 Qwen3 架构**: FlexInfer 的 4096 对齐和架构定义不支持 Qwen3-Next 的 512 专家 MoE。解决: 技术路线转向 upstream llama.cpp, 从最新 llama.cpp backport Qwen3 架构定义, 自行实现 SLIM-ARC 修改。
- 2. **GGML_CPU_REPACK 内存翻倍**: llama.cpp CPU 后端的 Q4_K → q4_K_8x8 重打包导致匿名内存 45→90 GB, 是 80B 在受限环境 OOM 的根因。解决: -DGGML_CPU_REPACK=OFF 禁用, 内存减半。
- 3. **并行实验导致内存竞争**: 三组消融并行跑导致 cgroup 内存竞争, 数据不可复现 (同一配置 tg8 从 0.28 到 1.03)。解决: 全部核心实验改为串行冷启动 (drop_caches + 单任务), 数据可信度恢复。
- 4. **src/llama-upstream 代码因.gitignore 误加而丢失**: WSL 重启后丢失全部 upstream 修改。解决: 开发 scripts/apply-slim-arc.py 幂等集成脚本, 所有 SLIM-ARC 修改通过 patches/ 可完整恢复, 并修正.gitignore。
- 5. **数据拼接与计算错误**: 审计发现 tab:ablation_single 的 Full 行拼接了两个不同日志的数值, MADV 贡献百分比计算方向错误 (-41% 应为 -29%), 32 GB baseline 误用 8 GB 数据。解决: 统一数据源到 logs/ablation/full-rerun/, 重算所有百分比, 重跑 32 GB baseline。
- 6. **Speculative Decoding 负收益**: llama.cpp 内置 ngram-simple speculative decoding 在 80B MoE 上 -53.2%。分析原因 (无 small draft model、MoE router 高熵、验证开销大), 诚实记录为负面结果, 未强行使用。

7.4 分工与协作

表 30: 团队分工

成员	角色	主要工作
欧阳易芃	项目负责人 / 系统优化	架构设计、SLIM-ARC 代码实现、实验调优、报告撰写
马福泉	实验评估	cgroups 环境搭建、benchmark 脚本、消融实验、数据收集
刘昊	文档与展示	文档维护、展示网站、图表制作、资料调研

指导教师: 赵帅、张献伟。

协作方式: 通过 Git 进行版本管理 (GitHub 公开仓库 + GitLab 比赛托管), 每日同步进展。使用 AI Agent (Claude Code) 辅助代码实现和文档生成, 所有 AI 参与均在 commit message 和文档中声明。指导教师定期 review 技术方向和实验设计。

7.5 提交仓库目录与文件描述

提交仓库 (GitLab: T2026105589911358/project3136859-389100) 的目录结构如下:

```
SLIM-ARC/  
src/llama-upstream/      # upstream llama.cpp + SLIM-ARC 修改  
src/slim-arc-*.h/.cpp    # SLIM-ARC 独立模块 (6 个文件)
```

```
scripts/
  apply-slim-arc.py      # 幂等集成脚本
  env/setup-cgroups.sh  # 三档 cgroups 配置
  bench/                # benchmark 脚本
patches/llama-upstream/ # SLIM-ARC 补丁（可恢复）
docs/
  design/               # 工程级代码文档（6 个）
  official/            # 赛题与官方材料
  papers/              # 参考论文与调研
reports/Competition_Report/ # 学术报告（LaTeX + PDF）
config/slim-arc.toml    # 配置文件
data/benchmarks/gsm8k/  # GSM8K 测试集
logs/                  # 实验日志（可溯源）
tests/                 # 环境测试
README.md              # 项目说明
LICENSE                # MIT
```

关键文件说明：scripts/apply-slim-arc.py 是幂等集成脚本，在干净 upstream 上运行即可恢复全部 SLIM-ARC 修改；patches/ 保存独立模块源码，作为 apply 脚本的备份；logs/ablation/full-rerun/ 存放串行冷启动实验日志，报告所有数据可溯源至此；reports/Competition_Report/main.pdf 为本报告 PDF。

7.6 比赛收获

- ▶ **技术收获**：深入理解了操作系统虚拟内存机制（mmap、madvise、page fault、cgroups）与大模型推理访存行为的协同。核心 insight 是 MADV_RANDOM 作为 MoE 稀疏性的天然 OS 接口——关闭顺序预读后，page fault 天然实现了“仅加载激活专家”。这一发现将 OS 课程知识（虚拟内存）与前沿系统问题（端侧 LLM）深度结合。
- ▶ **方法论收获**：建立了优化技术的 ABC 分类法（内核协同 / 量化压缩 / 计算融合），揭示了“A 释放 → B 利用 → C 加速”的级联协同关系。学会了对负面结果（prefetch 冗余、speculative 负收益）的诚实记录而非掩盖，这是学术可信度的基石。
- ▶ **工程收获**：掌握了幂等集成脚本（apply-slim-arc.py）保证第三方代码修改可恢复的工程实践；建立了串行冷启动实验协议确保数据可复现；通过两轮数据审计建立了数据溯源到原始日志的质量控制流程。
- ▶ **团队收获**：探索了 AI Agent（Claude Code）作为“Agent Programmer”辅助科研的协作模式，在代码实现、文档生成、数据审计中显著提升效率，同时通过 commit message 声明和人工 review 保证了学术诚信。

我们相信，操作系统虚拟内存机制与 MoE 稀疏性的深度协同，是解决端侧大模型推理“内存墙”问题的关键路径，这一思路对未来的端侧 Agent 系统设计具有参考价值。

7.7 AI 工具使用声明

在本项目的开发过程中，我们合理使用了 AI 辅助工具以提升工程效率。具体而言，AI 工具主要在以下方面提供了辅助：代码实现的样板生成与调试、文档的初稿撰写、实验数据的格式化整理，以及重复性脚本的编写。所有 AI 生成的内容均经过团队成员的人工审查、验证与修改，核心技术决策、系统架构设计、实验方案制定和学术判断均由团队成员独立完成。

我们始终坚持学术诚信原则：所有实验数据均来自真实的冷启动推理运行，可溯源至 logs/ 目录下的原始日志；所有引用的论文均经过团队成员阅读和核实；AI 工具的使用情况在 Git commit message 和项目文档中均有记录。项目的核心创新——利用 MADV_RANDOM 作为 MoE 稀疏性的操作系统接口——源于团队成员对虚拟内存机制与 MoE 架构的深入分析，是人工思考的成果，并非 AI 生成。

参考文献

- [1] H. Du, S. Wu, A. Kharlamova, N. Guan, and C. J. Xue, “Flexinfer: Breaking memory constraint via flexible and efficient offloading for on-device LLM inference,” in *Proceedings of the 5th Workshop on Machine Learning and Systems (EuroMLSys '25)*. ACM, 2025, pp. 56–65.
- [2] Y. Song, Z. Mi, H. Xie, and H. Chen, “PowerInfer: Fast large language model serving with a consumer-grade GPU,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, 2024.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “LLaMA: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [5] A. Yang *et al.*, “Qwen3 technical report,” 2025.
- [6] Qwen Team, “Qwen3-Next: Next-generation MoE language models,” <https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct>, 2024.
- [7] A. of the On-Device LLM Survey, “On-device large language models: A survey of model compression and system optimization,” *Artificial Intelligence Review*, 2026, accepted March 2026.
- [8] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, E. Hinton, and J. Dean, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [9] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity,” *Journal of Machine Learning Research*, vol. 23, pp. 1–39, 2022.
- [10] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” <https://github.com/ggml-org/llama.cpp>, 2024.
- [11] J. Chen, Z. Wang, K. Zhang, and Y. Meng, “MobileMoE: Scaling on-device mixture of experts,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [12] C. Wei *et al.*, “MoE-Prism: Disentangling monolithic experts for elastic MoE services via model-system co-designs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [13] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, “PowerInfer-2: Fast large language model inference on a smartphone,” in *Proceedings of the European Conference on Computer Systems (EuroSys)*, 2024.

- [14] H. Wang, S. Cao, Y. C. Lin, and G. Pekhimenko, “DUAL-BLADE: Dual-path NVMe-direct KV cache offloading for efficient large language model inference,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [15] A. of HillInfer, “HillInfer: Efficient long-context LLM inference on the edge with hierarchical KV eviction using SmartSSD,” in *Proceedings of the ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [16] A. of ScoutAttention, “ScoutAttention: Efficient KV cache offloading via layer-ahead CPU pre-computation for LLM inference,” in *Proceedings of the International Symposium on Computer Architecture (ISCA)*, 2024.
- [17] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 million context length LLM inference with KV cache quantization,” *arXiv preprint arXiv:2401.18079*, 2024.
- [18] Z. Liu, J. Ye, W. Li, Z. Yang, W. Xiong, L. Qiu, I. Lourentzou, T. Yu, Y. Hao, B. Guo, Y. Yang, R. Hou, C. Li, and B. He, “KIVI: A tuning-free asymmetric KV cache quantization protocol for llms,” *arXiv preprint arXiv:2402.02750*, 2024.
- [19] Y. Zhang, J. Wu, J. Flanagan, and W. Y. Wang, “ThinK: How to maintain long-context LLMs with limited memory,” *arXiv preprint arXiv:2407.21018*, 2024.
- [20] S. Tairin, S. Mahmud, H. Shen, and A. Iyer, “eMoE: Task-aware memory efficient mixture-of-experts-based (MoE) model inference,” in *arXiv preprint arXiv:2503.06823*, 2025.
- [21] X. Song, Z. Zhong, R. Chen, and H. Chen, “ProMoE: Fast MoE-based LLM serving using proactive caching,” *arXiv preprint arXiv:2410.22134*, 2025.
- [22] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [23] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *arXiv preprint arXiv:2307.08691*, 2023.
- [24] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [25] X. Zheng, D. Wei, J. Gao, Y. Song, Z. Mi, and H. Chen, “SolidAttention: Low-latency SSD-based serving on memory-constrained PCs,” in *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*, 2026.
- [26] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [27] The Open Group, “posix_madvise: Memory advisory (POSIX),” https://man7.org/linux/man-pages/man2/posix_madvise.2.html, 2024.

- [28] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [29] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” arXiv preprint arXiv:1609.07843, 2016.
- [30] Linux Kernel Documentation, “cgroup-v2: Linux Control Group v2,” <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html>, 2024.